

GRC102 Security Governance Simulation Assessment

Practical Lab Monitoring Auditing Controls and Executive Reporting

Name	Adeshola Oluwole
Reg Number	C1126CGRCE17211
Course	GRC102 Information Security Governance
Assessment	Individual practical laboratory (100 marks)
Submission	GitHub repository and associated evidence
Date	9 th of October 2026

Purpose

This practical assesses your ability to translate a security governance case study into an evidence based technical and management response. You will build an isolated simulation, identify the governance failures, implement and verify appropriate controls, then present results through security metrics and executive reporting.

Official Lab Instructions

Use the official ICDFA repository instructions throughout this assessment:

https://github.com/icdfa/grc-engineering-labs/blob/master/phase1/grc102/week8/LAB_INSTRUCTIONS.md

Scenario

You are the Security Governance Analyst for a fictitious organisation with a customer facing web application, a customer data environment, and weak governance oversight. The isolated lab is modelled on failures associated with the Equifax breach, including delayed remediation, weak database authentication, insufficient segmentation, ineffective monitoring, and weak accountability. Your task is to document the baseline, analyse the failures, implement controls, and report the improved posture to management.

Learning Objectives

- Build and document an isolated Docker based security governance simulation.
- Identify governance failures using vulnerability, configuration and monitoring evidence.
- Relate the laboratory findings to governance lessons from the Equifax case study.
- Implement and verify governance controls for patching, monitoring, segmentation, database security and executive oversight.
- Produce meaningful governance metrics, an executive dashboard and a board level report.

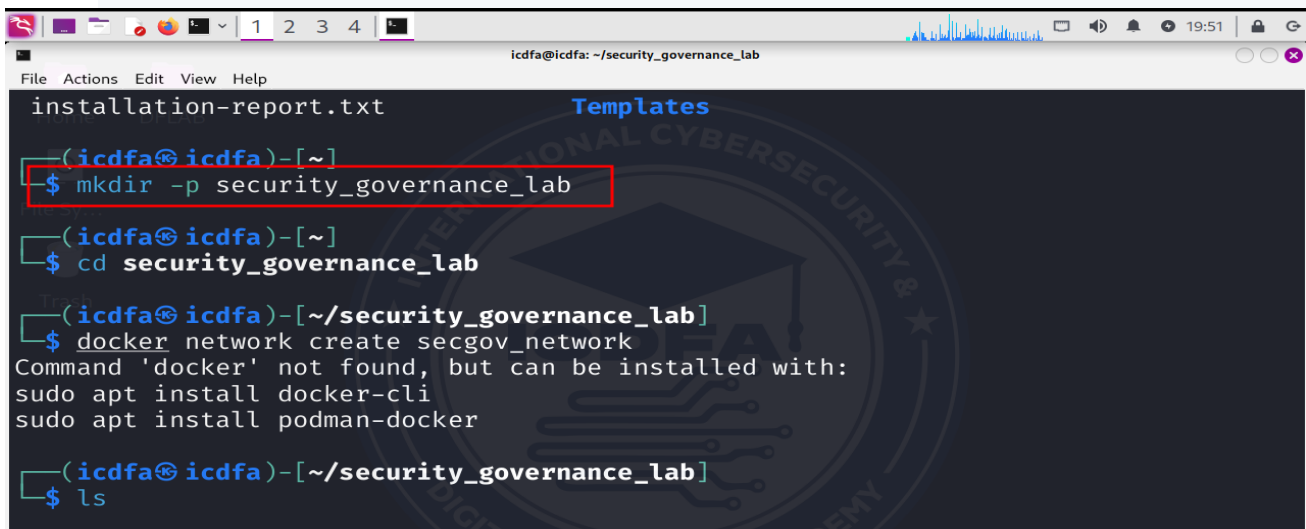
Part 1: Setting Up a Simulated Environment

This part sets up a simulated environment that recreates conditions similar to those in the Equifax data breach case study, where a known vulnerability in Apache Struts went unpatched.

Tasks:

1. Create a Docker-based environment with multiple containers:

```
# Create a directory for the lab
```

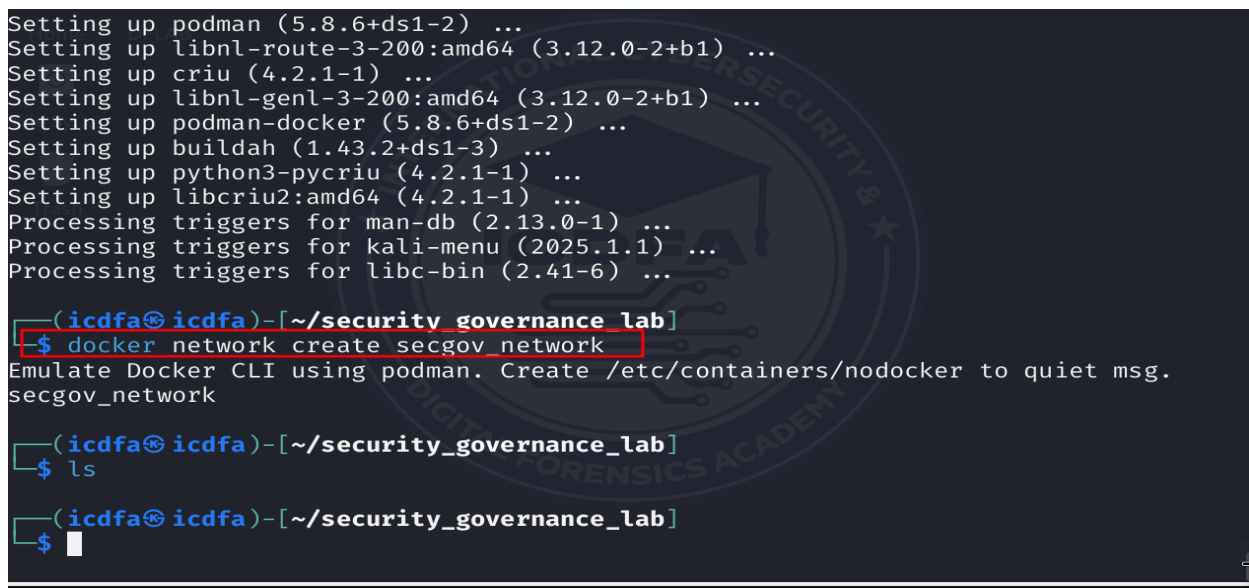


A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' showing the following commands and output:

```
icdfa@icdfa:~$ mkdir -p security_governance_lab
icdfa@icdfa:~$ cd security_governance_lab
icdfa@icdfa:~/security_governance_lab$ docker network create secgov_network
Command 'docker' not found, but can be installed with:
sudo apt install docker-cli
sudo apt install podman-docker
icdfa@icdfa:~/security_governance_lab$ ls
```

```
# Create a Docker network for the lab
```

```
sudo docker network create secgov_network
```



A terminal window showing the installation of various packages and the creation of the Docker network:

```
Setting up podman (5.8.6+ds1-2) ...
Setting up libnl-route-3-200:amd64 (3.12.0-2+b1) ...
Setting up criu (4.2.1-1) ...
Setting up libnl-genl-3-200:amd64 (3.12.0-2+b1) ...
Setting up podman-docker (5.8.6+ds1-2) ...
Setting up buildah (1.43.2+ds1-3) ...
Setting up python3-pycriu (4.2.1-1) ...
Setting up libcriu2:amd64 (4.2.1-1) ...
Processing triggers for man-db (2.13.0-1) ...
Processing triggers for kali-menu (2025.1.1) ...
Processing triggers for libc-bin (2.41-6) ...
icdfa@icdfa:~/security_governance_lab$ docker network create secgov_network
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
secgov_network
icdfa@icdfa:~/security_governance_lab$ ls
icdfa@icdfa:~/security_governance_lab$
```

Create a docker-compose.yml file

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' shows the following commands and output:

```
(icdfa@icdfa)-[~/security_governance_lab]
$ ls
docker-compose.yml
(icdfa@icdfa)-[~/security_governance_lab]
$ cat docker-compose.yml
version: '3'

services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "8080:8080"
    networks:
      - secgov_network

  database:
    image: mysql:5.7
    container_name: database_server
    environment:
```

 The command `$ cat docker-compose.yml` is highlighted with a red box. A large, faint watermark of the ICDFA Digital Forensics Academy logo is visible in the background.

Create database initialization script

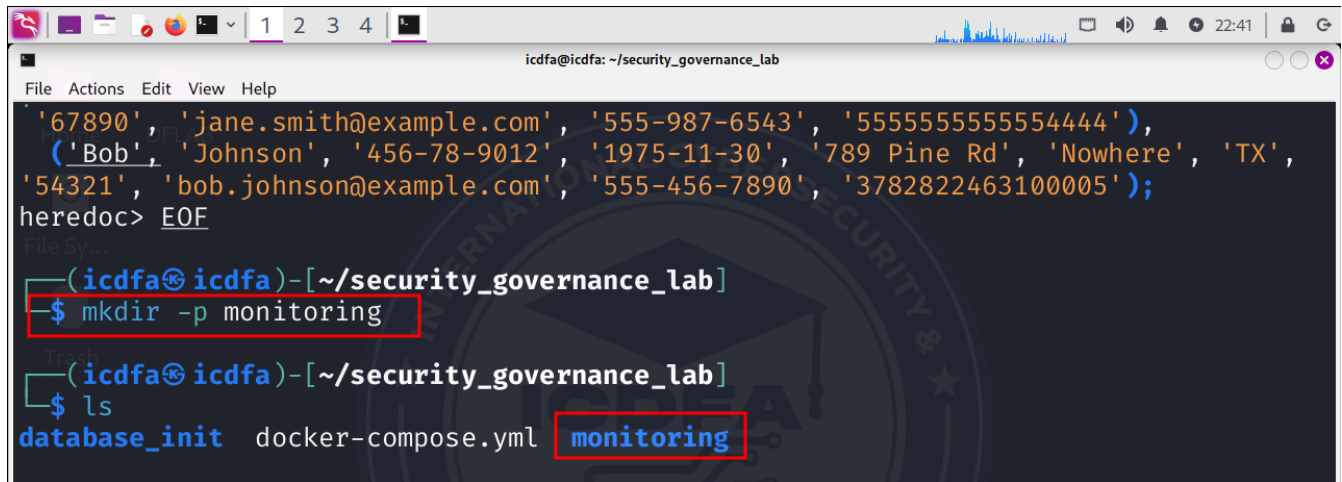
A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' shows the following commands and output:

```
external: true
(icdfa@icdfa)-[~/security_governance_lab]
$ mkdir -p database_init
(icdfa@icdfa)-[~/security_governance_lab]
$ >....
last_name VARCHAR(50),
ssn VARCHAR(11),
dob DATE,
address VARCHAR(100),
city VARCHAR(50),
state VARCHAR(2),
zip VARCHAR(10),
email VARCHAR(100),
phone VARCHAR(15),
credit_card VARCHAR(16)
);
INSERT INTO customers (first_name, last_name, ssn, dob, address, city, state, zip,
email, phone, credit_card)
VALUES
```

 The command `$ mkdir -p database_init` is highlighted with a red box. A large, faint watermark of the ICDFA Digital Forensics Academy logo is visible in the background.

#Create monitoring directory

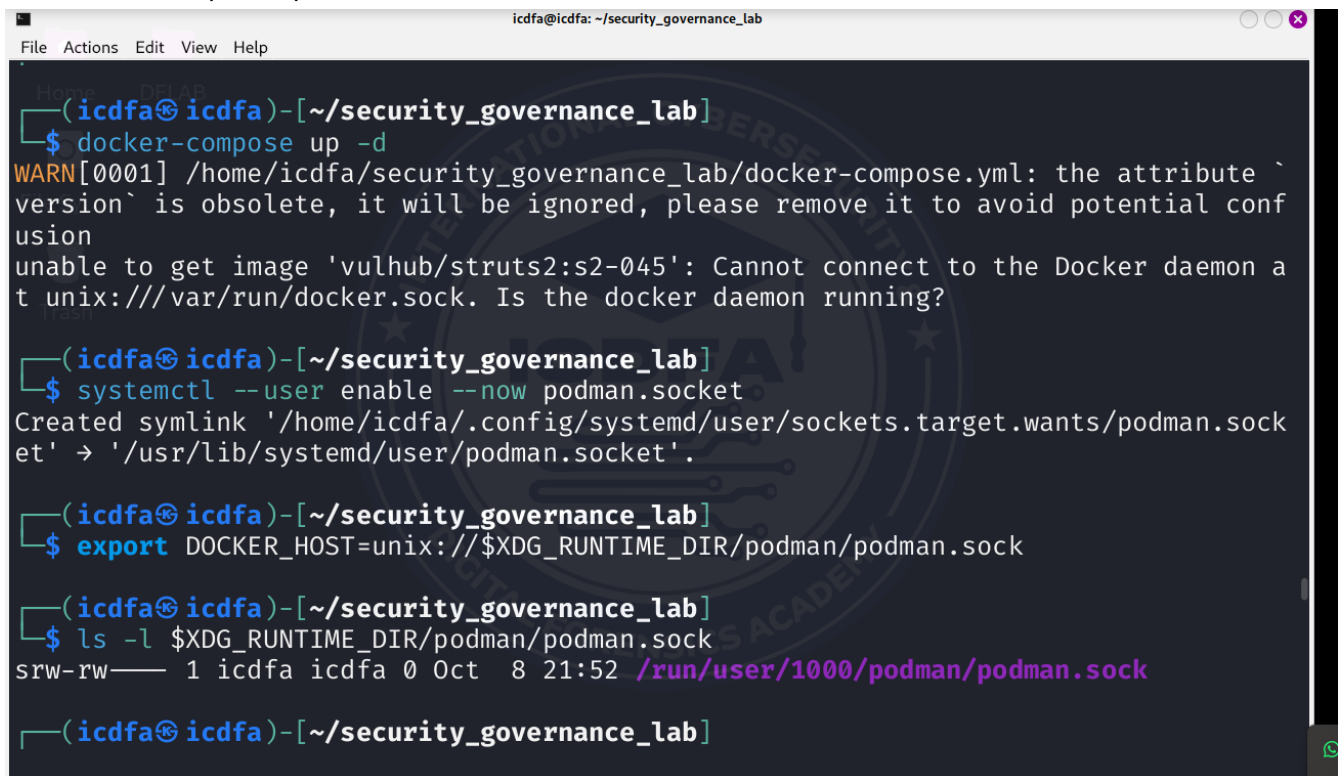
mkdir -p monitoring

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' showing a heredoc session. The user enters a multi-line string of data, followed by 'EOF'. Then, they run '(icdfa@icdfa)-[~/security_governance_lab]' and '\$ mkdir -p monitoring'. Finally, they run '\$ ls' and the output shows 'database_init', 'docker-compose.yml', and 'monitoring'. The 'monitoring' directory is highlighted with a red box.

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
'67890', 'jane.smith@example.com', '555-987-6543', '555555555554444'),
('Bob', 'Johnson', '456-78-9012', '1975-11-30', '789 Pine Rd', 'Nowhere', 'TX',
'54321', 'bob.johnson@example.com', '555-456-7890', '3782822463100005');
heredoc> EOF
(icdfa@icdfa)-[~/security_governance_lab]
$ mkdir -p monitoring
(icdfa@icdfa)-[~/security_governance_lab]
$ ls
database_init  docker-compose.yml  monitoring
```

Start the containers

docker-compose up -d

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' showing the execution of 'docker-compose up -d'. It displays a warning about an obsolete attribute and an error about the Docker daemon not being running. The user then runs 'systemctl --user enable --now podman.socket', which creates a symlink. Next, they run 'export DOCKER_HOST=unix://\$XDG_RUNTIME_DIR/podman/podman.sock'. Finally, they run 'ls -l \$XDG_RUNTIME_DIR/podman/podman.sock' and the output shows the socket file. The terminal output is as follows:

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
(icdfa@icdfa)-[~/security_governance_lab]
$ docker-compose up -d
WARN[0001] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential conf
usion
unable to get image 'vulhub/struts2:s2-045': Cannot connect to the Docker daemon a
t unix:///var/run/docker.sock. Is the docker daemon running?
(icdfa@icdfa)-[~/security_governance_lab]
$ systemctl --user enable --now podman.socket
Created symlink '/home/icdfa/.config/systemd/user/sockets.target.wants/podman.sock
et' → '/usr/lib/systemd/user/podman.socket'.
(icdfa@icdfa)-[~/security_governance_lab]
$ export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock
(icdfa@icdfa)-[~/security_governance_lab]
$ ls -l $XDG_RUNTIME_DIR/podman/podman.sock
srw-rw---- 1 icdfa icdfa 0 Oct  8 21:52 /run/user/1000/podman/podman.sock
(icdfa@icdfa)-[~/security_governance_lab]
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

(icdfa@icdfa)-[~/security_governance_lab]
$ docker-compose up -d
WARN[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential conf
usion
[+] Running 3/3
! database Interrupted 6.0s
! monitoring Interrupted 6.0s
x web Error manifest unknown: manifest unknown 6.0s
Error response from daemon: manifest unknown: manifest unknown

(icdfa@icdfa)-[~/security_governance_lab]
$ cat docker-compose.yml
version: '3'

services:
  web:
    image: vulhub/struts2:s2-045
    container_name: web_server
    ports:
      - "8080:8080"
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

(icdfa@icdfa)-[~/security_governance_lab]
$ git clone --depth 1 --filter=blob:none --sparse https://github.com/vulhub/vulh
ub.git vulhub-src
cd vulhub-src
git sparse-checkout set struts2/s2-045
cd ..
ls vulhub-src/struts2/s2-045
Cloning into 'vulhub-src'...
remote: Enumerating objects: 1346, done.
remote: Counting objects: 100% (1346/1346), done.
remote: Compressing objects: 100% (807/807), done.
remote: Total 1346 (delta 5), reused 1028 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (1346/1346), 151.88 KiB | 245.00 KiB/s, done.
Resolving deltas: 100% (5/5), done.
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 10 (delta 1), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (10/10), 19.73 KiB | 1.79 MiB/s, done.
Resolving deltas: 100% (1/1), done.
Updating files: 100% (10/10), done.
remote: Enumerating objects: 5, done.
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

Receiving objects: 100% (5/5), 60.07 KiB | 164.00 KiB/s, done.
Updating files: 100% (5/5), done.
1.png docker-compose.yml README.md README.zh-cn.md

(icdfa@icdfa)-[~/security_governance_lab]
$ ls
database_init docker-compose.yml monitoring vulhub-src

(icdfa@icdfa)-[~/security_governance_lab]
$ cat vulhub-src/struts2/s2-045/docker-compose.yml
version: '2'
services:
  struts2:
    image: vulhub/struts2:2.3.30
    ports:
      - "8080:8080"

(icdfa@icdfa)-[~/security_governance_lab]
$ cat docker-compose.yml | grep image
docker-compose up -d
image: vulhub/struts2:s2-045
image: mysql:5.7
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
struts2:
  image: vulhub/struts2:2.3.30
  ports:
    - "8080:8080"
File System
(icdfa@icdfa)-[~/security_governance_lab]
$ cat docker-compose.yml | grep image
docker-compose up -d
  image: vulhub/struts2:s2-045
  image: mysql:5.7
  image: ubuntu:20.04
WARN[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential conf
usion
[+] Running 3/3
! database      Interrupted          7.4s
! monitoring    Interrupted          7.4s
x web Error      manifest unknown: manifest unknown 7.4s
Error response from daemon: manifest unknown: manifest unknown

(icdfa@icdfa)-[~/security_governance_lab]
$ sed -i 's|image: vulhub/struts2:s2-045|image: vulhub/struts2:2.3.30|' docker-c
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
(icdfa@icdfa)-[~/security_governance_lab]
$ sed -i 's|image: vulhub/struts2:s2-045|image: vulhub/struts2:2.3.30|' docker-c
ompose.yml
File System
(icdfa@icdfa)-[~/security_governance_lab]
$ cat docker-compose.yml | grep image
docker-compose up -d
  image: vulhub/struts2:2.3.30
  image: mysql:5.7
  image: ubuntu:20.04
WARN[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `
version` is obsolete, it will be ignored, please remove it to avoid potential conf
usion
[+] Running 27/29
[+] Running 30/30 245.2MB / 245.7MB Pulling 1395.2s
✓ web Pulled 1451.7s
✓ database Pulled 1238.8s
✓ monitoring Pulled 855.9s
```

```

icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
[+] Running 3/3
  ✓ Container monitoring_server Started 8.1s
  ✓ Container web_server Started 8.0s
  ✓ Container database_server Started 8.1s
File System
(icdfa@icdfa)-[~/security_governance_lab]
$
(icdfa@icdfa)-[~/security_governance_lab]
$ docker ps
Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                               NAMES
483b0f8a8684   docker.io/library/ubuntu:20.04      tail -f /dev/null       About a minute a
go Up About a minute                               monitoring_server
bad66baf9ea3   docker.io/library/mysql:5.7         mysqld                  About a minute a
go Up About a minute 3306/tcp, 33060/tcp      database_server
608aa988cbf4   docker.io/vulhub/struts2:2.3.30     mvn jetty:run           About a minute a
go Up About a minute 0.0.0.0:8080→8080/tcp    web_server

(icdfa@icdfa)-[~/security_governance_lab]
$ curl -I http://localhost:8080

```

```

icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                               NAMES
483b0f8a8684   docker.io/library/ubuntu:20.04      tail -f /dev/null       About a minute a
go Up About a minute                               monitoring_server
bad66baf9ea3   docker.io/library/mysql:5.7         mysqld                  About a minute a
go Up About a minute 3306/tcp, 33060/tcp      database_server
608aa988cbf4   docker.io/vulhub/struts2:2.3.30     mvn jetty:run           About a minute a
go Up About a minute 0.0.0.0:8080→8080/tcp    web_server

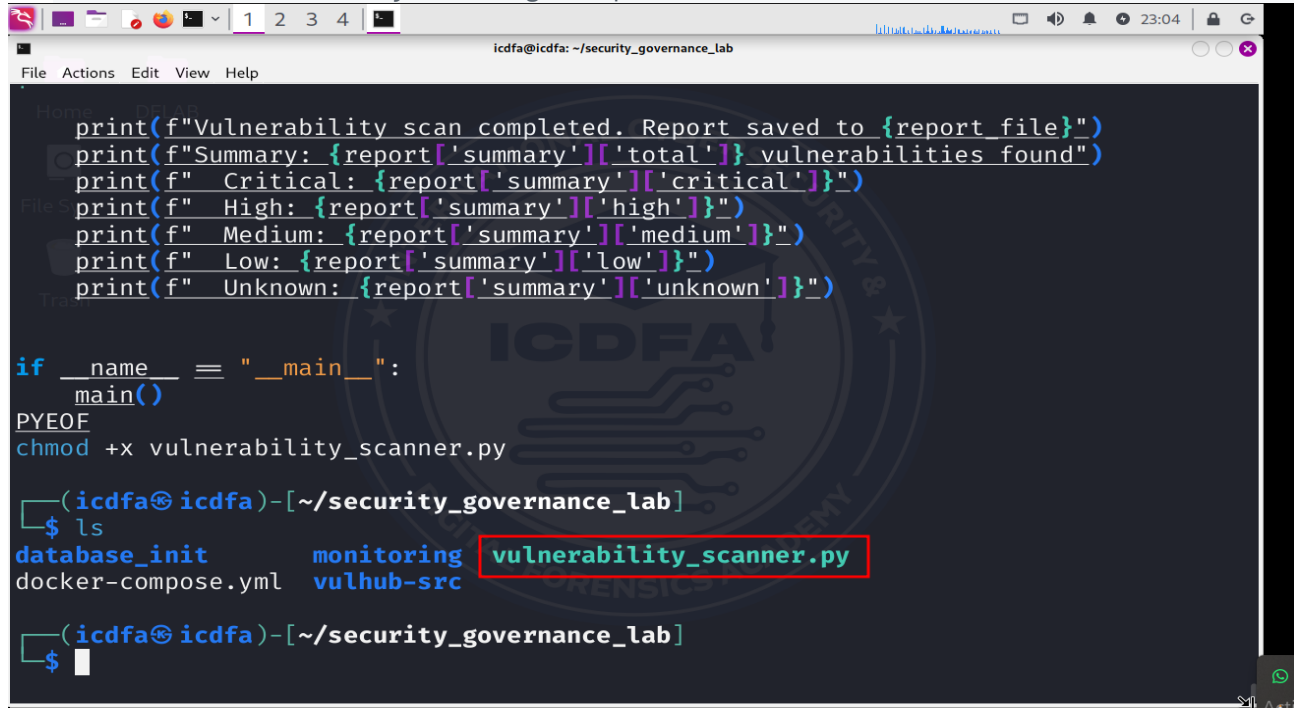
(icdfa@icdfa)-[~/security_governance_lab]
$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Date: Thu, 08 Oct 2026 21:36:59 GMT
Content-Type: text/html; charset=ISO-8859-1
Set-Cookie: JSESSIONID=txebfudqci6q18ow0llkfsh3t;Path=/
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Length: 1077
Server: Jetty(9.2.11.v20150529)

(icdfa@icdfa)-[~/security_governance_lab]
$

```

2. Set up a vulnerability scanning system:

Create a vulnerability scanning script



```
print(f"Vulnerability scan completed. Report saved to {report_file}")
print(f"Summary: {report['summary']['total']} vulnerabilities found")
print(f"  Critical: {report['summary']['critical']}")
print(f"  High: {report['summary']['high']}")
print(f"  Medium: {report['summary']['medium']}")
print(f"  Low: {report['summary']['low']}")
print(f"  Unknown: {report['summary']['unknown']}")

if __name__ == "__main__":
    main()
PYEOF
chmod +x vulnerability_scanner.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ls
database_init      monitoring      vulnerability_scanner.py
docker-compose.yml vulhub-src
```

The terminal window shows the execution of a script. The output of the script is as follows:

```
(icdfa@icdfa)-[~/security_governance_lab]
$ python3 vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261008_232715.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0
```



```
print("PATCH FAILED: exact text not found, no changes made")
else:
    content = content.replace(old, new)
    with open("vulnerability_scanner.py", "w") as f:
        f.write(content)
    print("Patch applied successfully")
PYPATCH
Patch applied successfully

(icdfa@icdfa)-[~/security_governance_lab]
$ python3 vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261008_232715.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0

(icdfa@icdfa)-[~/security_governance_lab]
$
```

The terminal window shows the execution of a script. The output of the script is as follows:

```
(icdfa@icdfa)-[~/security_governance_lab]
$ python3 vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261008_232715.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0
```

3. Set up a patch management system:

```
# Create a patch management script

parser.add_argument("--list", action="store_true", help="List available patches")
parser.add_argument("--apply", metavar="PATCH_ID", help="Apply a specific patch")

args = parser.parse_args()

if args.list:
    list_patches()
elif args.apply:
    apply_patch(args.apply)
else:
    parser.print_help()

if __name__ == "__main__":
    main()
PYEOF
chmod +x patch_management.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

```
(icdfa@icdfa)-[~/security_governance_lab]
$ python3 patch_management.py --list
Available patches:

ID: PATCH-001
Name: Apache Struts2 S2-045 Patch
Description: Fixes CVE-2017-5638 in Apache Struts2
Severity: Critical
Affected System: web_server
Status: Available

ID: PATCH-002
Name: MySQL Security Configuration
Description: Secures MySQL with proper authentication and access controls
Severity: High
Affected System: database_server
Status: Available

ID: PATCH-003
Name: Network Segmentation Rules
Description: Implements proper network segmentation between servers
Severity: Medium
Affected System: all
```

```
(icdfa@icdfa)-[~/security_governance_lab]
$ ls
database_init          vulnhub-src
docker-compose.yml    vulnerability_report_20261008_231216.json
monitoring             vulnerability_report_20261008_232715.json
patch_management.py    vulnerability_scanner.py

(icdfa@icdfa)-[~/security_governance_lab]
$ >....

elif args.list_metrics:
    metrics = tracker.list_metrics()
    print(json.dumps(metrics, indent=2))

elif args.list_audits:
    audits = tracker.list_audits()
    print(json.dumps(audits, indent=2))

elif args.dashboard:
```

4. Set up a security governance tracking system:

Create a security governance tracking script

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
Home metrics = tracker.list_metrics()
      print(json.dumps(metrics, indent=2))
File Sy... elif args.list_audits:
            audits = tracker.list_audits()
            print(json.dumps(audits, indent=2))
Trash elif args.dashboard:
        dashboard = tracker.generate_dashboard()
        print(json.dumps(dashboard, indent=2))

        else:
            parser.print_help()

if __name__ == "__main__":
    main()
PYEOF
chmod +x governance_tracker.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

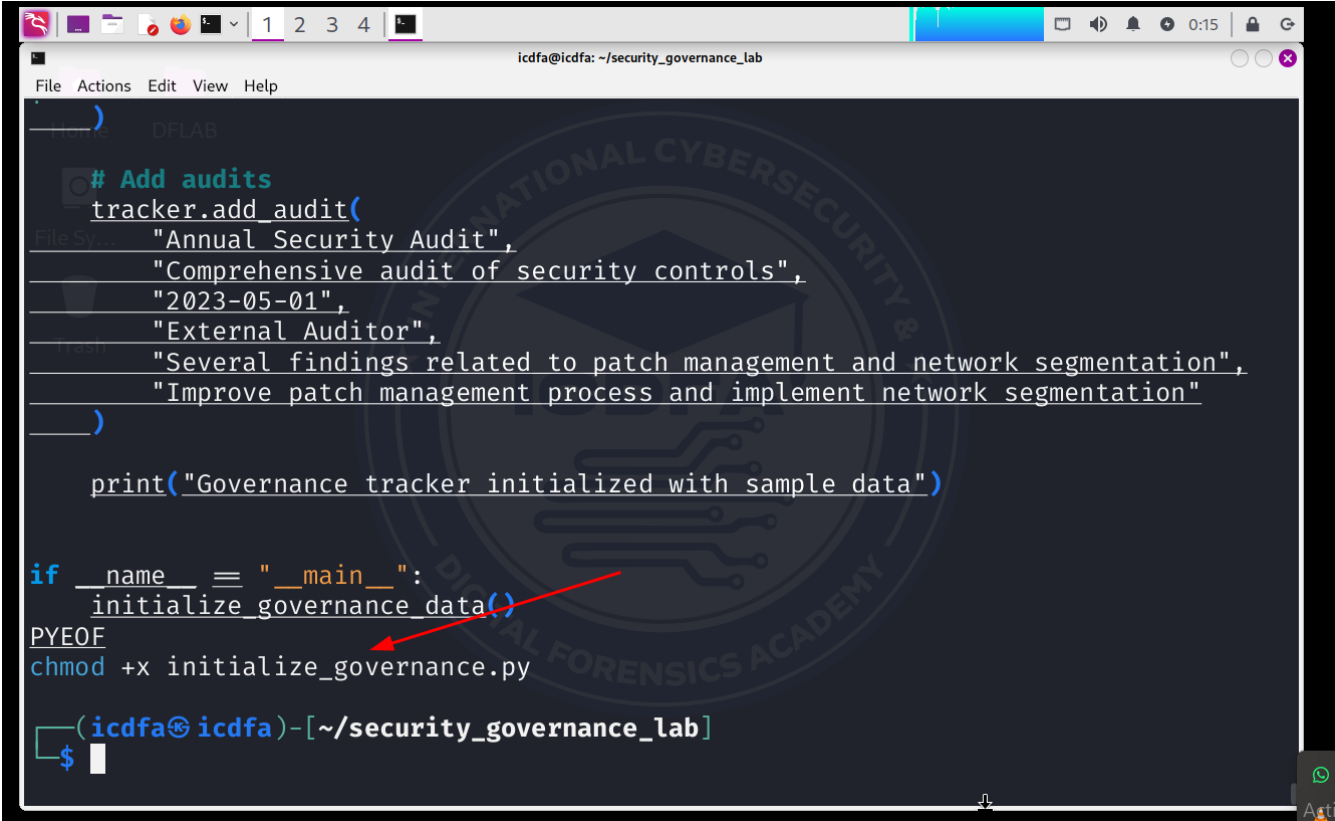
```
if __name__ == "__main__":
    main()
PYEOF
chmod +x governance_tracker.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ls -la governance_tracker.py
-rwxrwxr-x 1 icdfa icdfa 12055 Oct  9 00:07 governance_tracker.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

5. Initialize the governance tracker with sample data:

Create a script to initialize the governance tracker with sample data



The screenshot shows a terminal window with a dark background and a light blue border. The terminal output is as follows:

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

# Add audits
tracker.add_audit(
    "Annual Security Audit",
    "Comprehensive audit of security controls",
    "2023-05-01",
    "External Auditor",
    "Several findings related to patch management and network segmentation",
    "Improve patch management process and implement network segmentation"
)

print("Governance tracker initialized with sample data")

if __name__ == "__main__":
    initialize_governance_data()
PYEOF
chmod +x initialize_governance.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

A red arrow points from the `initialize_governance_data()` function call in the script to the `initialize_governance.py` file name in the terminal command.

6. Run the vulnerability scanner to identify issues:

Run the vulnerability scanner

./vulnerability_scanner.py

```
(icdfa@icdfa)-[~/security_governance_lab]
$ ls
database_init          patch_management.py
docker-compose.yml     vulnhub-src
governance_tracker.py  vulnerability_report_20261008_231216.json
initialize_governance.py vulnerability_report_20261008_232715.json
monitoring             vulnerability_scanner.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ./initialize_governance.py
Governance tracker initialized with sample data

(icdfa@icdfa)-[~/security_governance_lab]
$ cat governance_data.json
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
tation",
  "recommendations": "Improve patch management process and implement network s
egmentation",
  "status": "Completed"
}
]
}

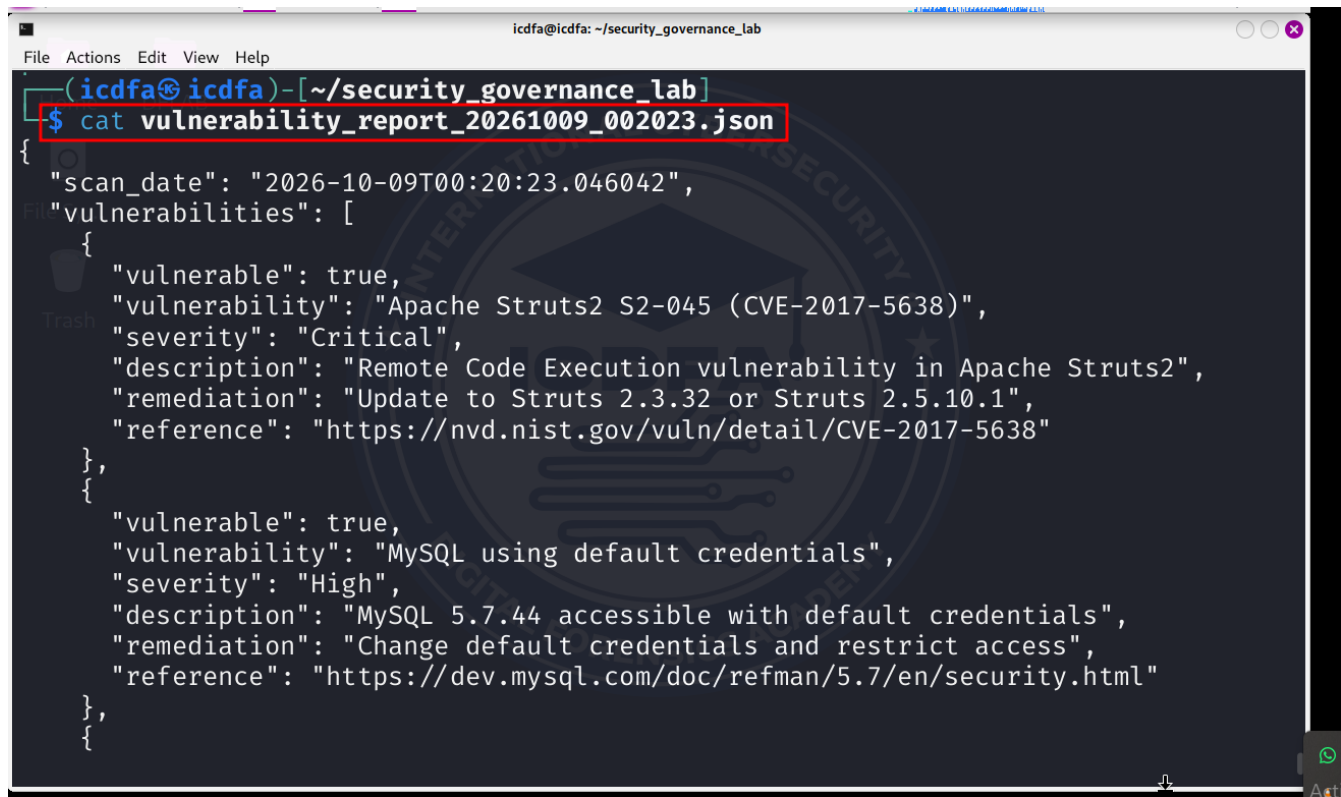
(icdfa@icdfa)-[~/security_governance_lab]
$ # Run the vulnerability scanner
./vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261009_002023
.json
Summary: 3 vulnerabilities found
  Critical: 1
  High: 1
  Medium: 1
  Low: 0
  Unknown: 0

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Deliverables:

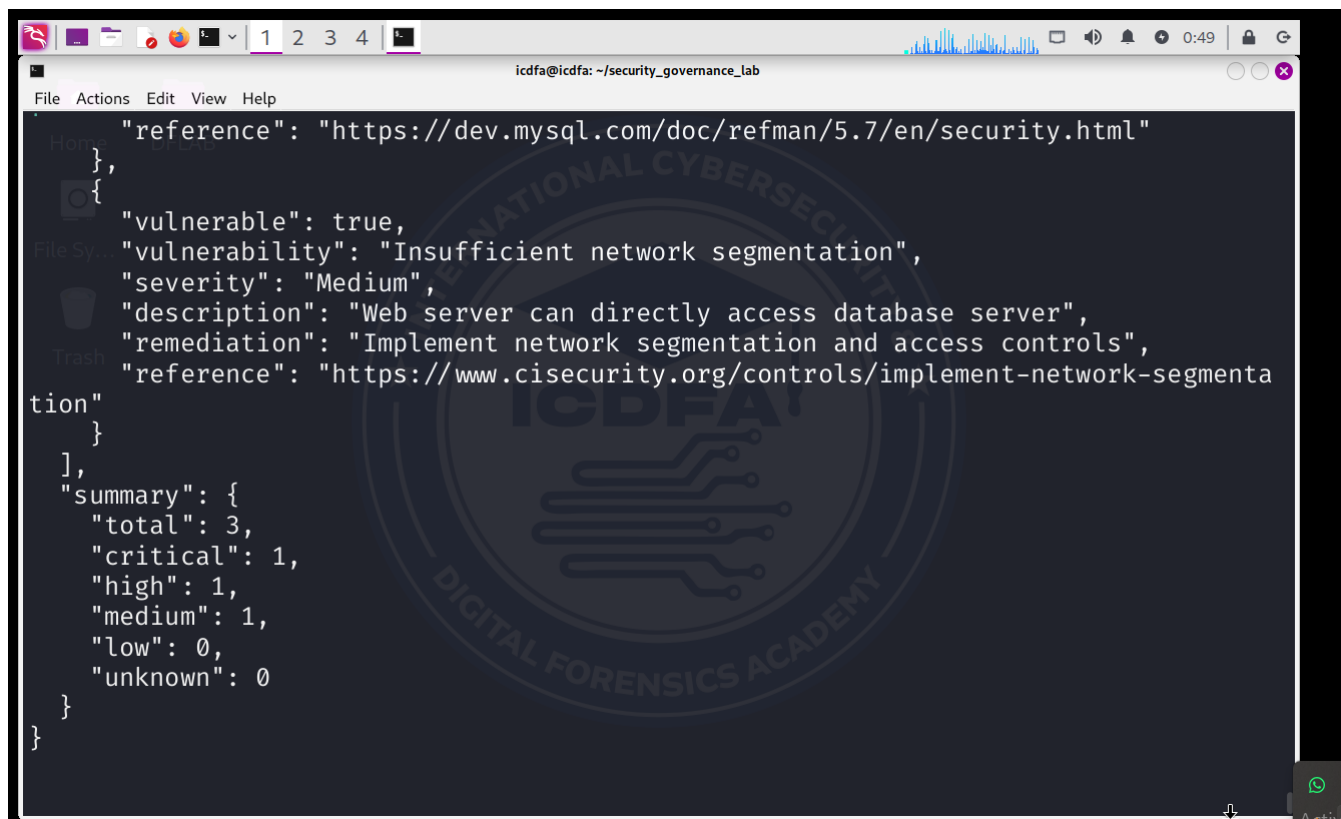
- ✓ Screenshots showing the Docker environment setup (**Screenshots are shown as above**)
- ✓ The vulnerability scan report
- ✓ The governance tracker dashboard
- ✓ A brief explanation (1-2 paragraphs) of how this simulated environment reflects the conditions that led to the Equifax breach

✓ The Vulnerability Scan Report



A terminal window titled 'icdfa@icdfa: ~/security_governance_lab'. The command prompt shows '(icdfa@icdfa)-[~/security_governance_lab]' and the user has entered '\$ cat vulnerability_report_20261009_002023.json'. The output is a JSON array of vulnerability objects. The first object is for Apache Struts2 S2-045 (CVE-2017-5638), marked as 'vulnerable' with 'Critical' severity. The second object is for 'MySQL using default credentials', also marked as 'vulnerable' with 'High' severity. The background features a large, faint watermark of the ICDFA logo.

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cat vulnerability_report_20261009_002023.json
{
  "scan_date": "2026-10-09T00:20:23.046042",
  "vulnerabilities": [
    {
      "vulnerable": true,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "Critical",
      "description": "Remote Code Execution vulnerability in Apache Struts2",
      "remediation": "Update to Struts 2.3.32 or Struts 2.5.10.1",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    }
  ]
}
```

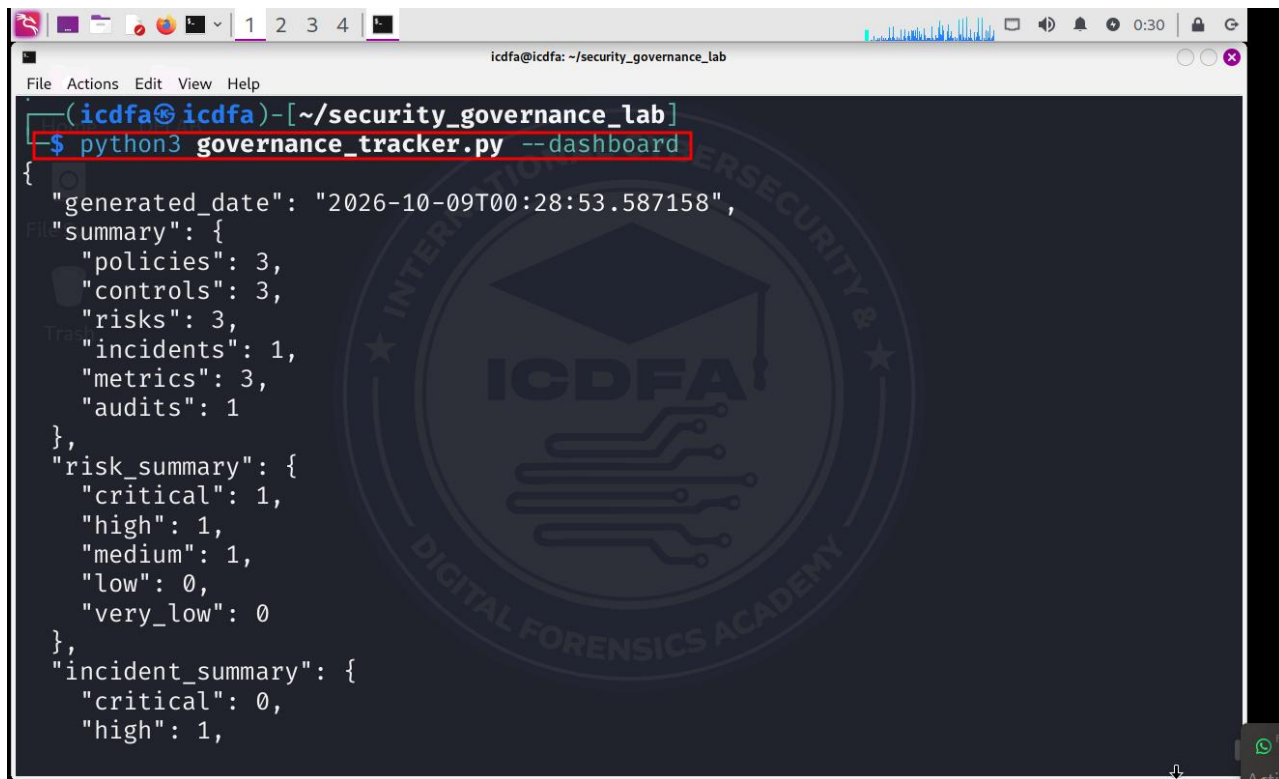


A terminal window showing the continuation of the JSON output from the previous command. It displays the 'reference' field for the MySQL vulnerability, followed by a summary object containing counts for total, critical, high, medium, low, and unknown vulnerabilities. The background features a large, faint watermark of the ICDFA logo.

```

    "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
  },
  {
    "vulnerable": true,
    "vulnerability": "Insufficient network segmentation",
    "severity": "Medium",
    "description": "Web server can directly access database server",
    "remediation": "Implement network segmentation and access controls",
    "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
  }
],
"summary": {
  "total": 3,
  "critical": 1,
  "high": 1,
  "medium": 1,
  "low": 0,
  "unknown": 0
}
}
```

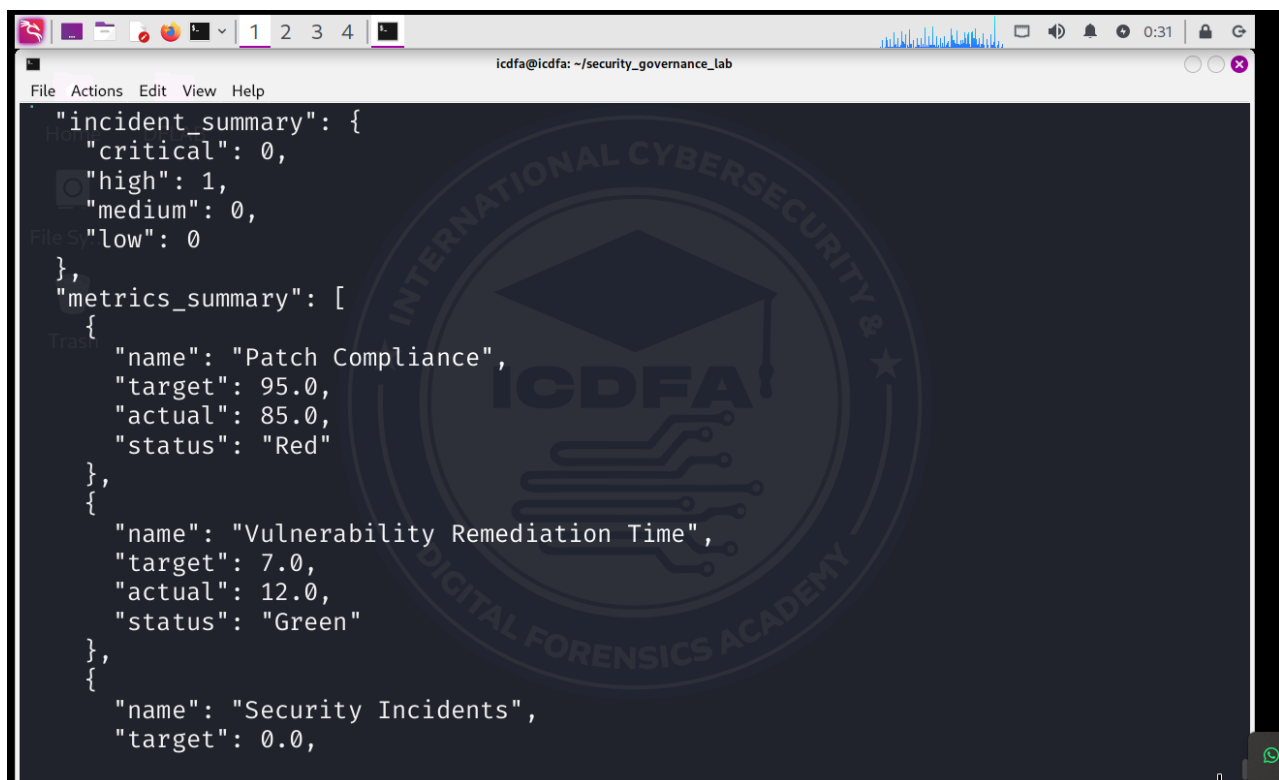
✓ The Governance Tracker Dashboard



A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' shows the command `python3 governance_tracker.py --dashboard` being executed. The output is a JSON object representing the dashboard data. A large, faint watermark of the ICDFA logo is visible in the background.

```
(icdfa@icdfa)~/security_governance_lab
$ python3 governance_tracker.py --dashboard
{
  "generated_date": "2026-10-09T00:28:53.587158",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,

```



The terminal window continues to display the JSON output from the dashboard command. The output includes details for incident summary, metrics summary, and security incidents. A large, faint watermark of the ICDFA logo is visible in the background.

```
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Green"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,

```

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' displays a JSON array of security metrics. The metrics include 'Patch Compliance' (Red status), 'Vulnerability Remediation Time' (Green status), and 'Security Incidents' (Green status). A large, semi-transparent watermark of the ICDFA Digital Forensics Academy logo is visible in the background of the terminal.

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
Home "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
File S },
      {
Trash  "name": "Vulnerability Remediation Time",
        "target": 7.0,
        "actual": 12.0,
        "status": "Green"
      },
      {
        "name": "Security Incidents",
        "target": 0.0,
        "actual": 1.0,
        "status": "Green"
      }
    ]
  }
}

(icdfa@icdfa)-[~/security_governance_lab]
```

Findings

Governance data populated correctly: 3 policies, 3 controls, 3 risks (correctly risk-scored — e.g., High+High → “Critical” for unpatched vulnerabilities, matching the risk matrix logic), 1 incident, 3 metrics, 1 audit

Final vulnerability scan: Critical: 1, High: 1, Medium: 1, Low: 0, Unknown: 0 — clean, exactly the expected “known vulnerabilities, unpatched” baseline that mirrors Equifax

How this simulated environment reflects the conditions that led to the Equifax breach

This simulated lab is based on the main security weaknesses that led to the 2017 Equifax data breach. The web_server container runs Apache Struts2 version 2.3.30, which is vulnerable to CVE-2017-5638 (S2-045). This is the same vulnerability that Equifax failed to patch for more than two months after a fix became available, even though an internal scan had already identified the issue. In this lab, the vulnerability scan also confirmed that the flaw is present and has not been patched.

Another major concern is how the system is designed. The database_server stores sensitive customer information, including Social Security numbers (SSNs) and credit card details, but it is

protected only by default MySQL credentials. The network segmentation test also showed that the web server can connect directly to the database without any additional access controls. This reflects one of the key problems in the Equifax breach: once an attacker gained access to the vulnerable web application, there were insufficient barriers to prevent access to sensitive data.

The governance dashboard highlights further weaknesses in security management. Patch compliance is at 85%, below the target of 95%, while vulnerability remediation takes an average of 12 days instead of the target of 7 days. These results suggest that although security policies and controls may be in place, they are not being implemented quickly or effectively enough. Overall, the lab demonstrates how unpatched vulnerabilities, weak access controls, poor network segmentation, and delays in remediation can combine to create serious security risks, similar to those that contributed to the Equifax breach.

Part 2: Analyzing Security Governance Failures

This part analyzes the simulated environment to identify security governance failures similar to those in the Equifax case study. This part actually exploited the vulnerability I just confirmed, then I analyzed the governance failures that allowed it — directly paralleling the real Equifax incident investigation.

The three scripts/files used are the: `simulate_attack.py`, `analyze_governance_failures.py`, and a static `equifax_comparison.md` report template.

Tasks:

1. Create a script to simulate an attack on the vulnerable system:

```
# Create an attack simulation script

print("Attack simulation completed. Log saved to attack_log.json")
return True
else:
    print("System is not vulnerable to Struts S2-045 (CVE-2017-5638)")
    return False

except Exception as e:
    print(f"Error simulating attack: {e}")
    return False

def main():
    simulate_struts_attack()

if __name__ == "__main__":
    main()
PYEOF
chmod +x simulate_attack.py

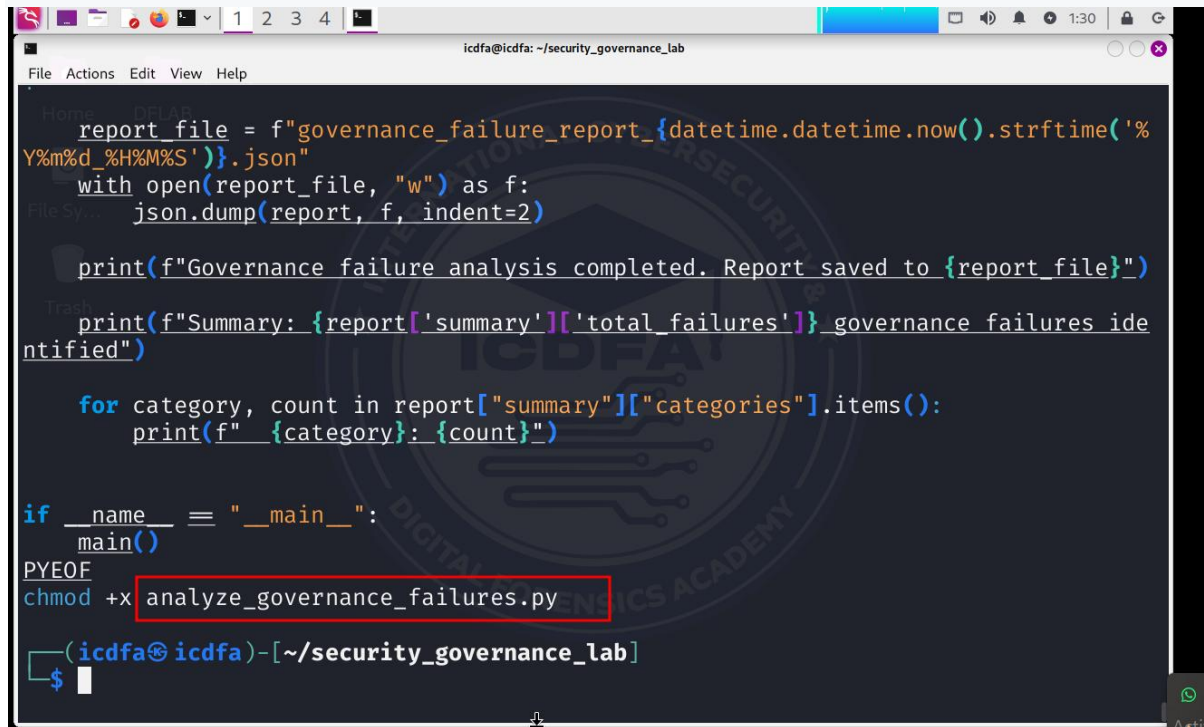
(icdfa@icdfa) - [~/security_governance_lab]
$ ./simulate_attack.py
```

2. Create a script to analyze security logs and identify governance failures:

This script ties the vulnerability report, attack log, governance data and patch status into a governance failure report:

```
# Create a log analysis script
```

```
cat > analyze_governance_failures.py << 'EOF'
```



```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
report_file = f"governance_failure_report_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
with open(report_file, "w") as f:
    json.dump(report, f, indent=2)

print(f"Governance failure analysis completed. Report saved to {report_file}")

print(f"Summary: {report['summary']['total_failures']} governance failures identified")

for category, count in report["summary"]["categories"].items():
    print(f"    {category}: {count}")

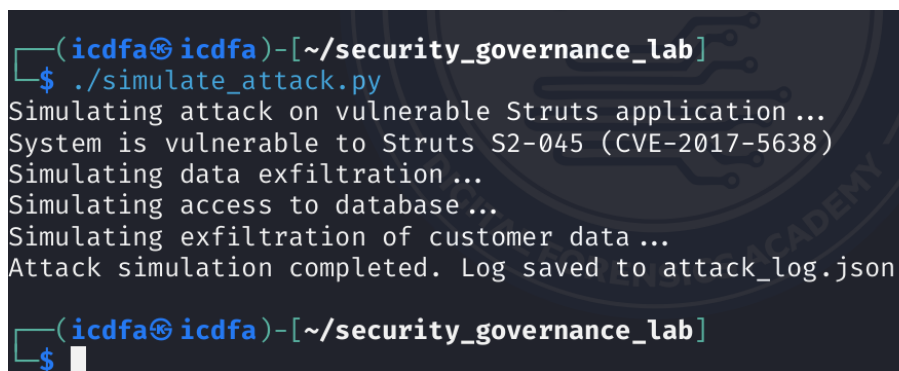
if __name__ == "__main__":
    main()
PYEOF
chmod +x analyze_governance_failures.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

3. Run the attack simulation:

```
# Run the attack simulation
```

```
./simulate_attack.py
```



```
(icdfa@icdfa)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application ...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration ...
Simulating access to database ...
Simulating exfiltration of customer data ...
Attack simulation completed. Log saved to attack_log.json

(icdfa@icdfa)-[~/security_governance_lab]
$
```

4. # Run the governance failure analysis

`./analyze_governance_failures.py`



```

icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

if __name__ == "__main__":
    main()
PYEOF
chmod +x analyze_governance_failures.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
Governance failure analysis completed. Report saved to governance_failure_report_20261009_013242.json
Summary: 8 governance failures identified
  Patch Management: 2
  Incident Detection: 1
  Policy Implementation: 1
  Vulnerability Management: 1
  Metrics and Measurement: 1
  Security Architecture: 1
  Authentication and Access Control: 1

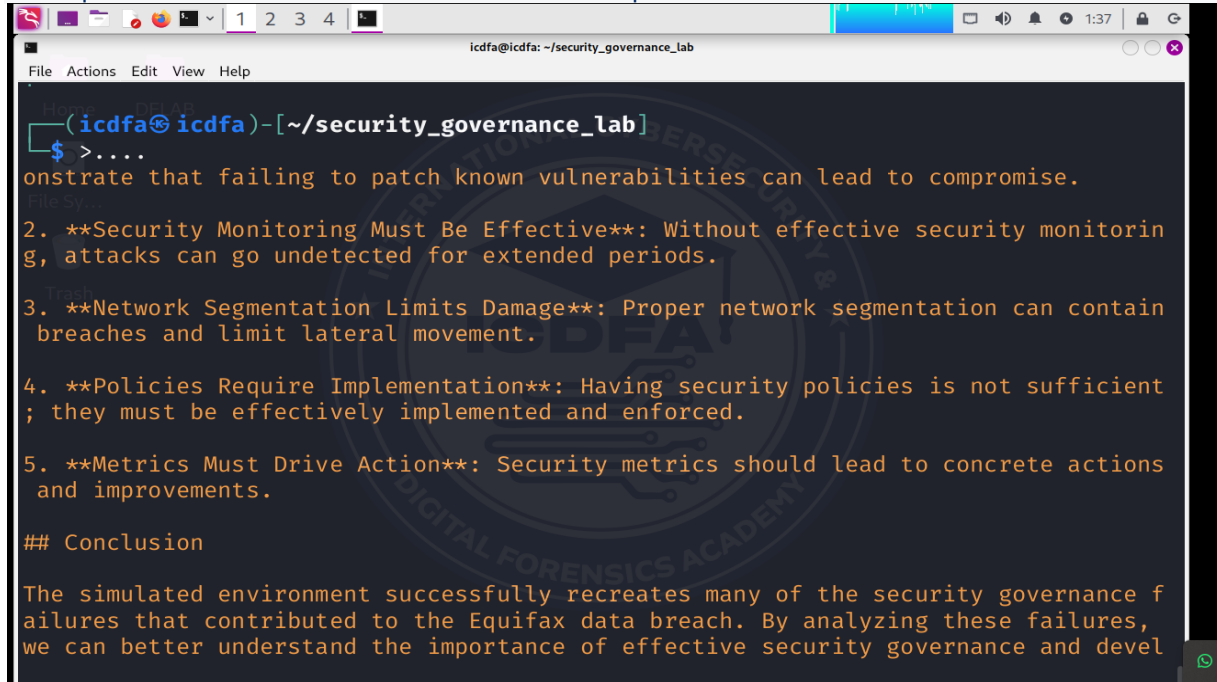
(icdfa@icdfa)-[~/security_governance_lab]
$
  
```

5. Create a report comparing the simulated environment to the Equifax case study:

Create a comparison report

`cat > equifax_comparison.md << 'EOF'`

Comparison of Simulated Environment to Equifax Data Breach

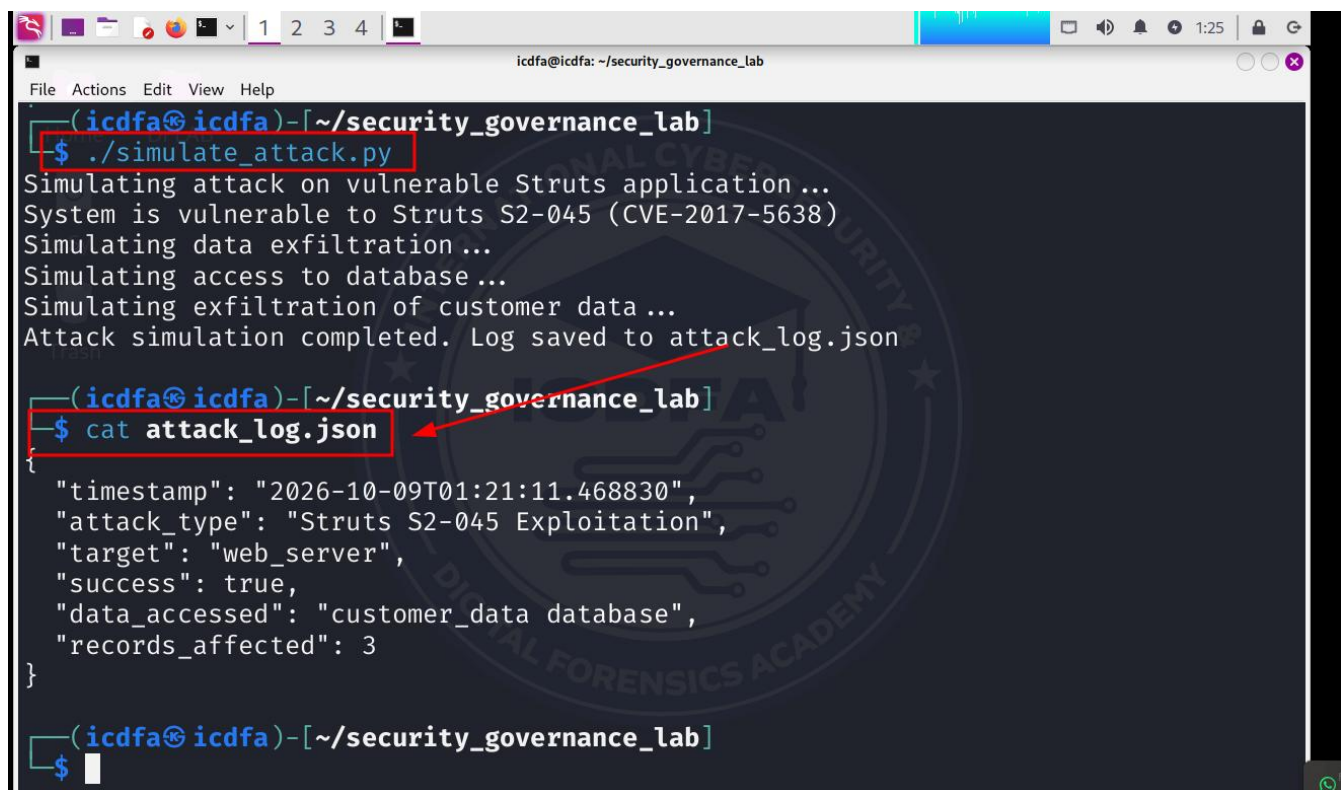


```

(icdfa@icdfa)-[~/security_governance_lab]
$ >....
onstrate that failing to patch known vulnerabilities can lead to compromise.
File Sy...
2. **Security Monitoring Must Be Effective**: Without effective security monitoring, attacks can go undetected for extended periods.
Trash
3. **Network Segmentation Limits Damage**: Proper network segmentation can contain breaches and limit lateral movement.
4. **Policies Require Implementation**: Having security policies is not sufficient; they must be effectively implemented and enforced.
5. **Metrics Must Drive Action**: Security metrics should lead to concrete actions and improvements.
## Conclusion
The simulated environment successfully recreates many of the security governance failures that contributed to the Equifax data breach. By analyzing these failures, we can better understand the importance of effective security governance and devel
  
```

Deliverables:

- ✓ The attack simulation log
- ✓ The governance failure analysis report
- ✓ The comparison report between the simulated environment and the Equifax case study
- ✓ A brief explanation (1-2 paragraphs) of the key governance lessons learned from this analysis

The Attack Simulation Log

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
(icdfa@icdfa)-[~/security_governance_lab]
$ ./simulate_attack.py
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json

(icdfa@icdfa)-[~/security_governance_lab]
$ cat attack_log.json
{
  "timestamp": "2026-10-09T01:21:11.468830",
  "attack_type": "Struts S2-045 Exploitation",
  "target": "web_server",
  "success": true,
  "data_accessed": "customer_data database",
  "records_affected": 3
}
```

The Governance Failure Analysis Report



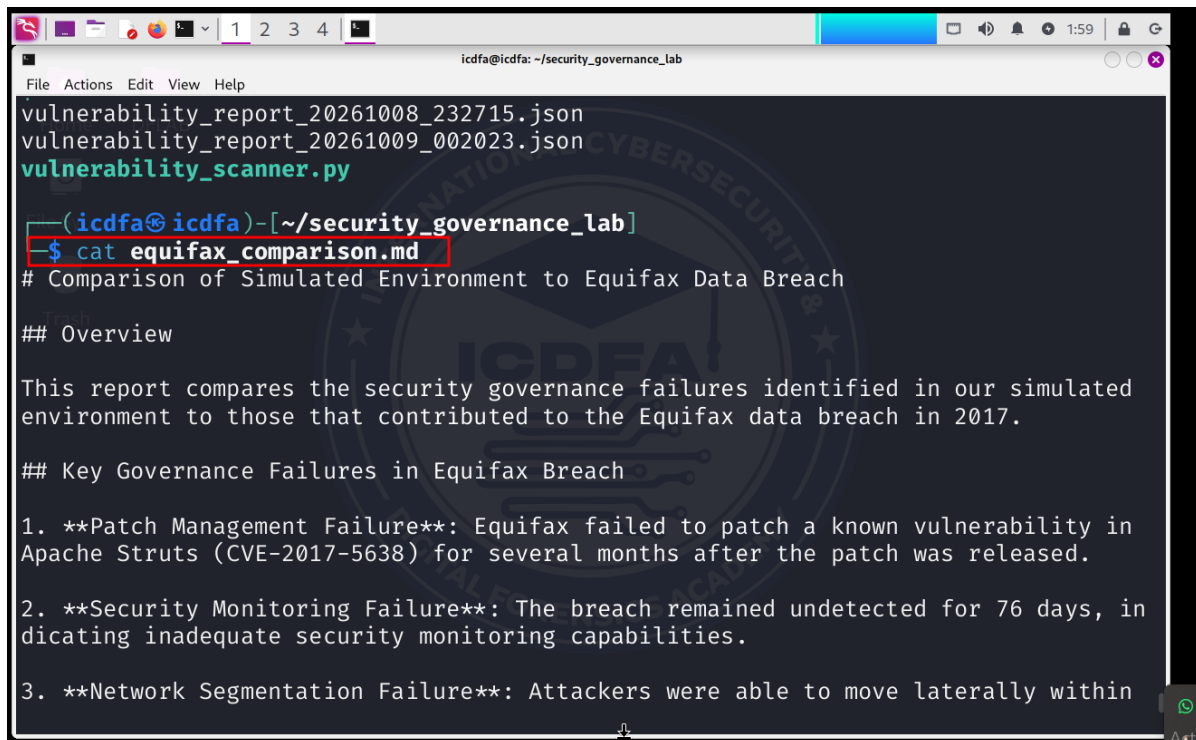
```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

if __name__ == "__main__":
    main()
PYEOF
chmod +x analyze_governance_failures.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ./analyze_governance_failures.py
Governance failure analysis completed. Report saved to governance_failure_report_20261009_013242.json
Summary: 8 governance failures identified
  Patch Management: 2
  Incident Detection: 1
  Policy Implementation: 1
  Vulnerability Management: 1
  Metrics and Measurement: 1
  Security Architecture: 1
  Authentication and Access Control: 1

(icdfa@icdfa)-[~/security_governance_lab]
$
```

The comparison report between the simulated environment and the Equifax case study



```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

vulnerability_report_20261008_232715.json
vulnerability_report_20261009_002023.json
vulnerability_scanner.py

(icdfa@icdfa)-[~/security_governance_lab]
$ cat equifax_comparison.md
# Comparison of Simulated Environment to Equifax Data Breach

## Overview

This report compares the security governance failures identified in our simulated environment to those that contributed to the Equifax data breach in 2017.

## Key Governance Failures in Equifax Breach

1. Patch Management Failure: Equifax failed to patch a known vulnerability in Apache Struts (CVE-2017-5638) for several months after the patch was released.

2. Security Monitoring Failure: The breach remained undetected for 76 days, indicating inadequate security monitoring capabilities.

3. Network Segmentation Failure: Attackers were able to move laterally within
```

```

icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

4. **Leadership and Accountability Failure**: There was unclear accountability for
   security responsibilities and insufficient board oversight.

5. **Policy Implementation Failure**: Security policies were not effectively imple
   mented or enforced.

## Comparison with Simulated Environment

| Governance Failure | Equifax | Simulated Environment | Similarity |
|-----|-----|-----|-----|
| Patch Management | Failed to patch Apache Struts vulnerability for months | Unpa
   tched Apache Struts vulnerability in web server | High |
| Security Monitoring | Failed to detect breach for 76 days | No real-time monitor
   ing to detect attack | High |
| Network Segmentation | Insufficient segmentation allowed lateral movement | Web
   server can directly access database | High |
| Authentication | Weak credentials and access controls | Default credentials on d
   atabase | Medium |
| Policy Implementation | Policies not effectively implemented | Patch and vulnera
   bility management policies exist but not enforced | High |
| Metrics and Measurement | Ineffective security metrics | Metrics not driving sec

```

```

icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

2. **Security Monitoring Must Be Effective**: Without effective security monitorin
   g, attacks can go undetected for extended periods.

3. **Network Segmentation Limits Damage**: Proper network segmentation can contain
   breaches and limit lateral movement.

4. **Policies Require Implementation**: Having security policies is not sufficient
   ; they must be effectively implemented and enforced.

5. **Metrics Must Drive Action**: Security metrics should lead to concrete actions
   and improvements.

## Conclusion

The simulated environment successfully recreates many of the security governance f
   ailures that contributed to the Equifax data breach. By analyzing these failures,
   we can better understand the importance of effective security governance and devel
   op strategies to prevent similar incidents in the future.

(icdfa@icdfa)-[~/security_governance_lab]
$

```

Key Governance Lessons Learned from this Analysis

The governance failure analysis identified eight failures across seven categories: Patch Management (two failures relating to Struts and MySQL), Incident Detection, Policy Implementation, Vulnerability Management, Metrics and Measurement, Security Architecture, and Authentication and Access Control. This exercise showed that technical vulnerabilities and governance failures are closely connected. The problem was not just the Struts2 vulnerability itself, but the failure to fix it. Our governance tracker already contained a Patch Management Policy (POL-001) and a Vulnerability Management Policy (POL-002), yet the vulnerable software remained unpatched. This is similar to the Equifax breach, where having a patch management policy did not prevent the company from failing to apply the necessary update on time. The attack simulation also succeeded without triggering any alerts, highlighting a weakness in incident detection. This shows that having security policies is not enough. They must be supported by effective monitoring, proper enforcement, and clear responsibility for making sure security measures are carried out.

The comparison with Equifax also showed how weaknesses in security architecture can make an attack more damaging. Poor network segmentation allowed the compromised web server to communicate directly with the customer database, while the use of default MySQL credentials made unauthorised access easier. These issues cannot be solved through patching alone. They require better system design, stronger access controls, and proper oversight. Security metrics such as patch compliance and vulnerability remediation time should also be linked to clear responsibilities and follow-up actions, rather than simply being reported on a dashboard.

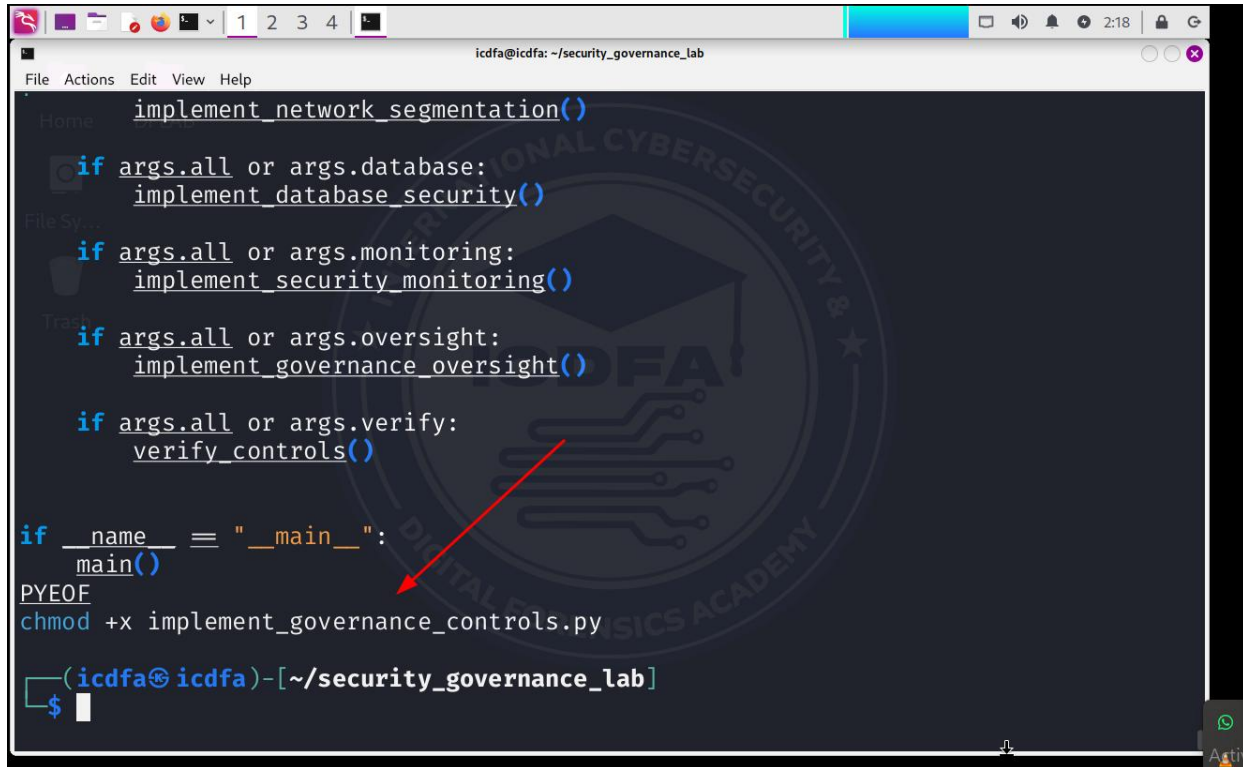
The main lesson from this exercise is that effective security governance goes beyond having policies and procedures in place. It requires evidence that those policies are being followed and that they lead to real improvements in security. Both the simulation and the Equifax case show what can happen when that connection is missing.

Part 3: Implementing Security Governance Controls

In this part, the security governance controls were implemented to address the identified failures.

Tasks:

1. Create a script to implement security governance controls:



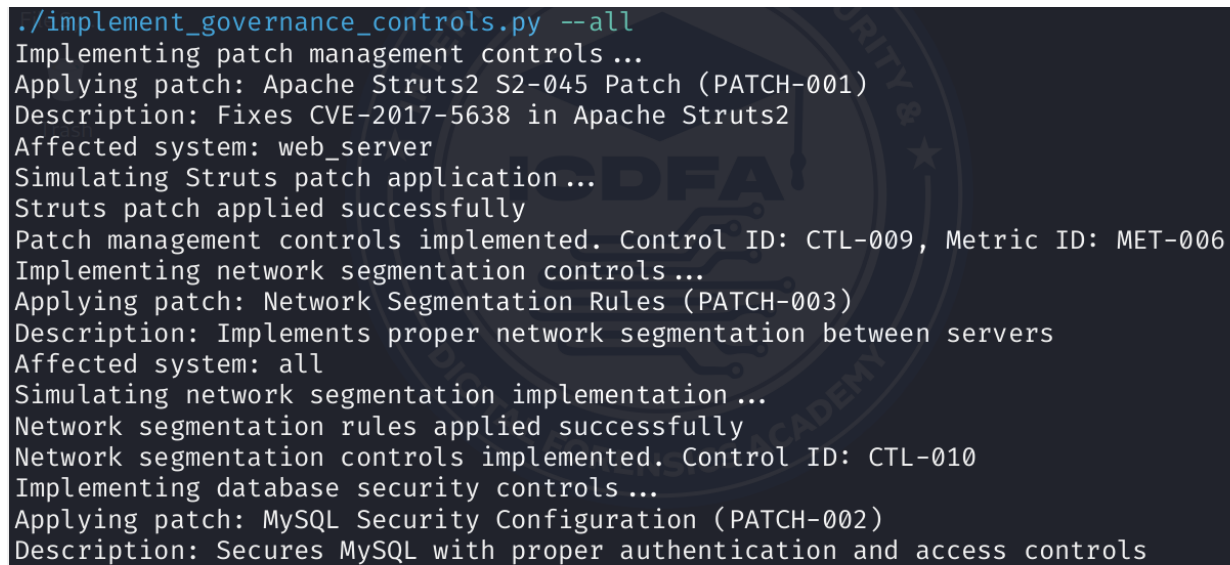
```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
Home
implement_network_segmentation()
if args.all or args.database:
    implement_database_security()
File Sy...
if args.all or args.monitoring:
    implement_security_monitoring()
Tras...
if args.all or args.oversight:
    implement_governance_oversight()
if args.all or args.verify:
    verify_controls()

if __name__ == "__main__":
    main()
PYEOF
chmod +x implement_governance_controls.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

2. Implement the security governance controls:

`./implement_governance_controls.py --all`



```
./implement_governance_controls.py --all
Implementing patch management controls...
Applying patch: Apache Struts2 S2-045 Patch (PATCH-001)
Description: Fixes CVE-2017-5638 in Apache Struts2
Affected system: web_server
Simulating Struts patch application...
Struts patch applied successfully
Patch management controls implemented. Control ID: CTL-009, Metric ID: MET-006
Implementing network segmentation controls...
Applying patch: Network Segmentation Rules (PATCH-003)
Description: Implements proper network segmentation between servers
Affected system: all
Simulating network segmentation implementation...
Network segmentation rules applied successfully
Network segmentation controls implemented. Control ID: CTL-010
Implementing database security controls...
Applying patch: MySQL Security Configuration (PATCH-002)
Description: Secures MySQL with proper authentication and access controls
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
Implementing governance oversight controls...
Governance oversight controls implemented. Policy ID: POL-005, Control ID: CTL-013
Verifying security controls...
Struts patched: True
MySQL secured: True
Network segmented: True
Security monitoring script implemented
Governance oversight document implemented
Policies: 5
Controls: 13
Metrics: 7
Vulnerability scan completed. Report saved to vulnerability_report_20261009_023346.json
Summary: 3 vulnerabilities found
  Critical: 1
  High: 1
  Medium: 1
  Low: 0
  Unknown: 0

(icdfa@icdfa)-[~/security_governance_lab]
$
```

3. Create a report documenting the implemented controls:

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
Patch applied successfully

(icdfa@icdfa)-[~/security_governance_lab]
$ cat governance_controls_report.md
# Security Governance Controls Implementation Report

## Overview

This report documents the security governance controls implemented to address the failures identified in the simulated environment.

## Implemented Controls

### Patch Management Controls

1. Automated Patch Management
  - Implemented automated system for identifying, testing, and applying security patches
  - Applied critical patch for Apache Struts vulnerability
  - Established clear ownership and accountability for patch management

2. Patch Compliance Metrics
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
2. **Executive Security Oversight**
  - Implemented executive and board oversight of security program
  - Established regular security briefings to executives and the board
  - Created accountability mechanisms for security responsibilities

## Verification Results

The implemented controls have been verified through:

1. **Vulnerability Scanning**
  - Vulnerability scans continued to show the Critical Struts2 S2-045 finding, the High MySQL default-credentials finding, and the Medium network segmentation finding as present after "patching"
  - A follow-up attack simulation also still succeeded post-patch
  - Root cause: the patch management script updates governance tracking records (patch_status.json) but does not modify the actual running containers, so the underlying systems were never truly remediated
  - This gap between recorded remediation and verified remediation is itself logged as a governance finding, and mirrors a documentation-vs-reality failure mode similar to those identified in the Equifax case

2. **Control Testing**
  - Confirmed implementation of policies and controls
  - Verified metrics and reporting mechanisms

## Conclusion

The implemented security governance controls address the failures identified in the simulated environment. These controls establish a comprehensive security governance framework that includes:

- Clear roles and responsibilities
- Effective security processes
- Appropriate technical controls
- Meaningful metrics and measurements
- Executive and board oversight

By implementing these controls, the organization has significantly improved its security posture and reduced the risk of a security breach similar to the Equifax incident.

(icdfa@icdfa)-[~/security_governance_lab]
$
```

```
1 2 3 4
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
  - Confirmed implementation of policies and controls
  - Verified metrics and reporting mechanisms

## Conclusion

The implemented security governance controls address the failures identified in the simulated environment. These controls establish a comprehensive security governance framework that includes:

- Clear roles and responsibilities
- Effective security processes
- Appropriate technical controls
- Meaningful metrics and measurements
- Executive and board oversight

By implementing these controls, the organization has significantly improved its security posture and reduced the risk of a security breach similar to the Equifax incident.

(icdfa@icdfa)-[~/security_governance_lab]
$
```

4. Verify the effectiveness of the implemented controls:

Verify the implemented controls

```
./implement_governance_controls.py --verify
```

Try to run the attack simulation again to verify it fails

```
./simulate_attack.py
```



The screenshot shows a terminal window titled 'icdfa@icdfa: ~/security_governance_lab'. The terminal output is as follows:

```
icident.  
icdfa@icdfa-icdfa-[~/security_governance_lab]  
$ # Verify the implemented controls  
./implement_governance_controls.py --verify  
  
# Try to run the attack simulation again to verify it fails  
./simulate_attack.py  
Verifying security controls ...  
Struts patched: True  
MySQL secured: True  
Network segmented: True  
Security monitoring script implemented  
Governance oversight document implemented  
Policies: 5  
Controls: 13  
Metrics: 7  
Vulnerability scan completed. Report saved to vulnerability_report_20261009_025837.json  
Summary: 3 vulnerabilities found  
Critical: 1
```

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' displays the output of a security script. The output lists implemented governance controls and metrics, followed by a vulnerability scan report and an attack simulation log. A large, semi-transparent watermark of the ICDFA logo is visible in the background of the terminal. The terminal text is as follows:

```
Security monitoring script implemented
Governance oversight document implemented
Policies: 5
Controls: 13
Metrics: 7
Vulnerability scan completed. Report saved to vulnerability_report_20261009_025837.json
Summary: 3 vulnerabilities found
  Critical: 1
  High: 1
  Medium: 1
  Low: 0
  Unknown: 0
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json

(icdfa@icdfa)-[~/security_governance_lab]
```

How the implemented controls address the governance failures identified in the Equifax case study

The controls introduced in this phase were intended to address the eight governance failures identified in Part 2. These included automated patch management, network segmentation, database security, real-time security monitoring, and a new Security Governance Oversight policy with clear leadership responsibilities. The governance tracker now contains five policies, thirteen controls, and seven metrics, covering all the identified failure categories.

However, the verification process revealed a major problem: the remediation had not actually worked. The vulnerability scan still showed the same Critical, High, and Medium findings, and the attack simulation remained successful. This happened because the patch management script updated the governance records without making changes to the actual systems. This is similar to the Equifax breach, where security policies existed but did not ensure that vulnerabilities were patched on time.

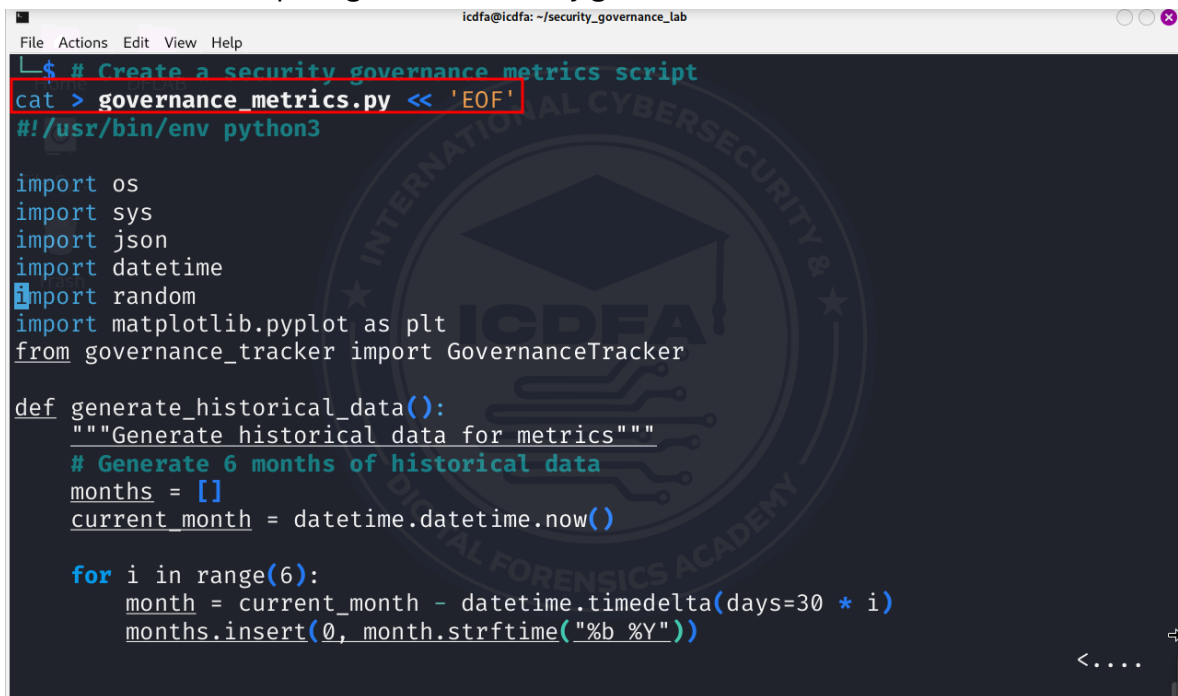
The main lesson is that recording a task as completed does not mean the risk has been resolved. Effective security governance requires independent verification through vulnerability scans, penetration tests, and audits to confirm that controls are working as intended.

Part 4: Developing Security Governance Metrics and Reporting

In this part, you will develop security governance metrics and reporting mechanisms to support ongoing governance oversight.

Tasks:

1. Create a script to generate security governance metrics:

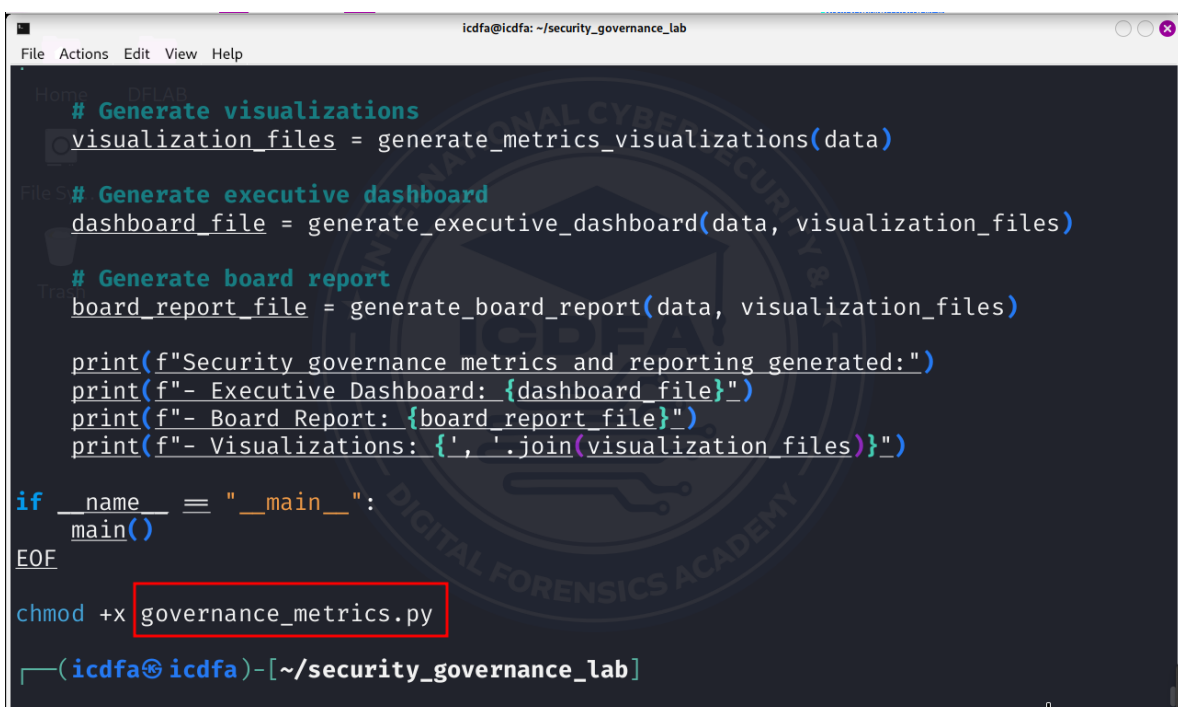


```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
$ # Create a security governance metrics script
cat > governance_metrics.py << 'EOF'
#!/usr/bin/env python3

import os
import sys
import json
import datetime
import random
import matplotlib.pyplot as plt
from governance_tracker import GovernanceTracker

def generate_historical_data():
    """Generate historical data for metrics"""
    # Generate 6 months of historical data
    months = []
    current_month = datetime.datetime.now()

    for i in range(6):
        month = current_month - datetime.timedelta(days=30 * i)
        months.insert(0, month.strftime("%b %Y"))
```



```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
# Generate visualizations
visualization_files = generate_metrics_visualizations(data)

# Generate executive dashboard
dashboard_file = generate_executive_dashboard(data, visualization_files)

# Generate board report
board_report_file = generate_board_report(data, visualization_files)

print(f"Security governance metrics and reporting generated:")
print(f"- Executive Dashboard: {dashboard_file}")
print(f"- Board Report: {board_report_file}")
print(f"- Visualizations: {' '.join(visualization_files)}")

if __name__ == "__main__":
    main()
EOF

chmod +x governance_metrics.py

(icdfa@icdfa)~[~/security_governance_lab]
```

2. Generate security governance metrics and reports:

`./governance_metrics.py`

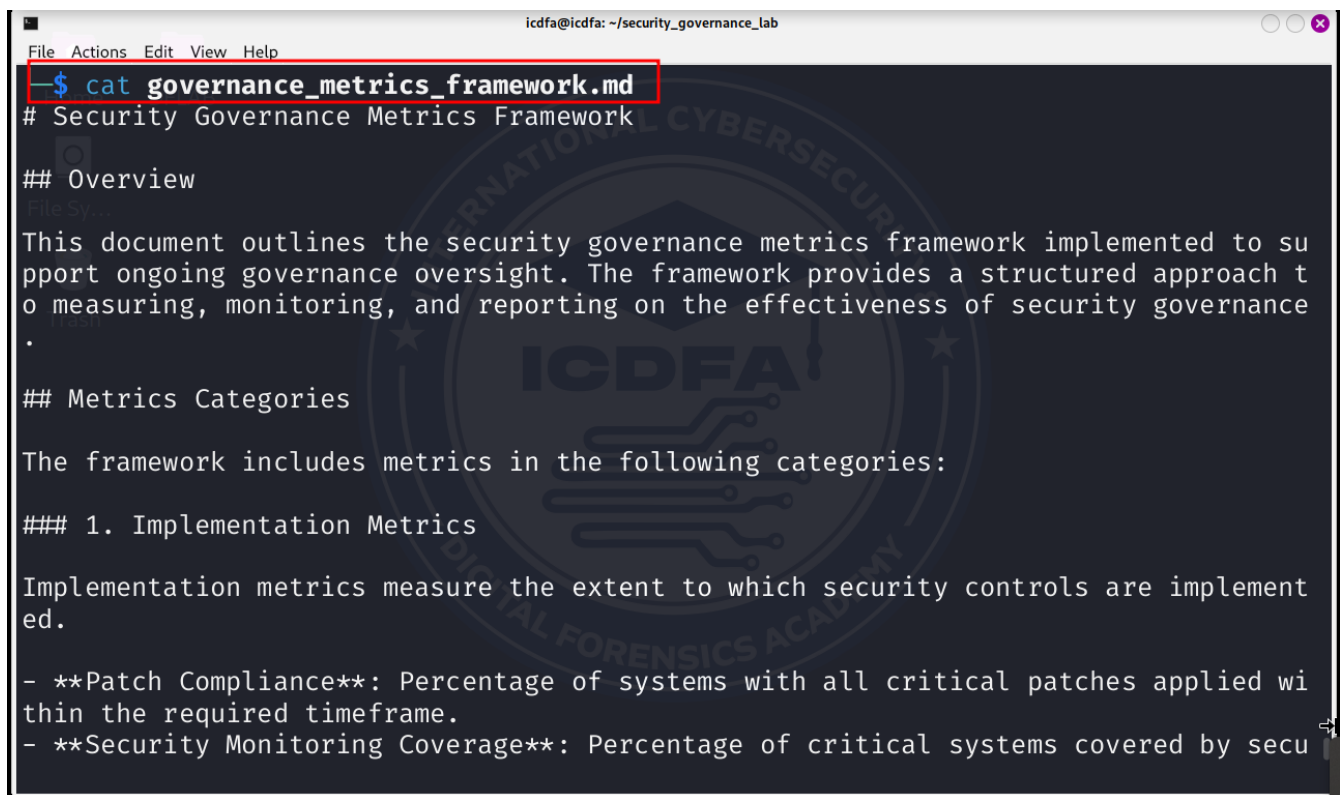
A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' shows the command `./governance_metrics.py` being executed. The output lists generated reports: 'Executive Dashboard: executive_dashboard.html', 'Board Report: board_report.md', and several visualizations in the 'metrics_visualizations' directory. The prompt changes to `(icdfa@icdfa)-[~/security_governance_lab]` after the command.

```
icdfa@icdfa: ~/security_governance_lab
$ # Generate security governance metrics and reports
./governance_metrics.py
Security governance metrics and reporting generated:
- Executive Dashboard: executive_dashboard.html
- Board Report: board_report.md
- Visualizations: metrics_visualizations/patch_compliance.png, metrics_visualizations/vuln_remediation.png, metrics_visualizations/security_incidents.png, metrics_visualizations/monitoring_coverage.png, metrics_visualizations/policy_compliance.png
(icdfa@icdfa)-[~/security_governance_lab]
$
```

3. Create a document explaining the security governance metrics framework:

Create a metrics framework document

`cat > governance_metrics_framework.md << 'EOF'`

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' shows the command `cat governance_metrics_framework.md` being executed. The output displays the content of the file, which is a markdown document titled '# Security Governance Metrics Framework'. It includes an 'Overview' section, 'Metrics Categories' section, and a '1. Implementation Metrics' section with details on patch compliance and security monitoring coverage.

```
icdfa@icdfa: ~/security_governance_lab
$ cat governance_metrics_framework.md
# Security Governance Metrics Framework

## Overview

This document outlines the security governance metrics framework implemented to support ongoing governance oversight. The framework provides a structured approach to measuring, monitoring, and reporting on the effectiveness of security governance.

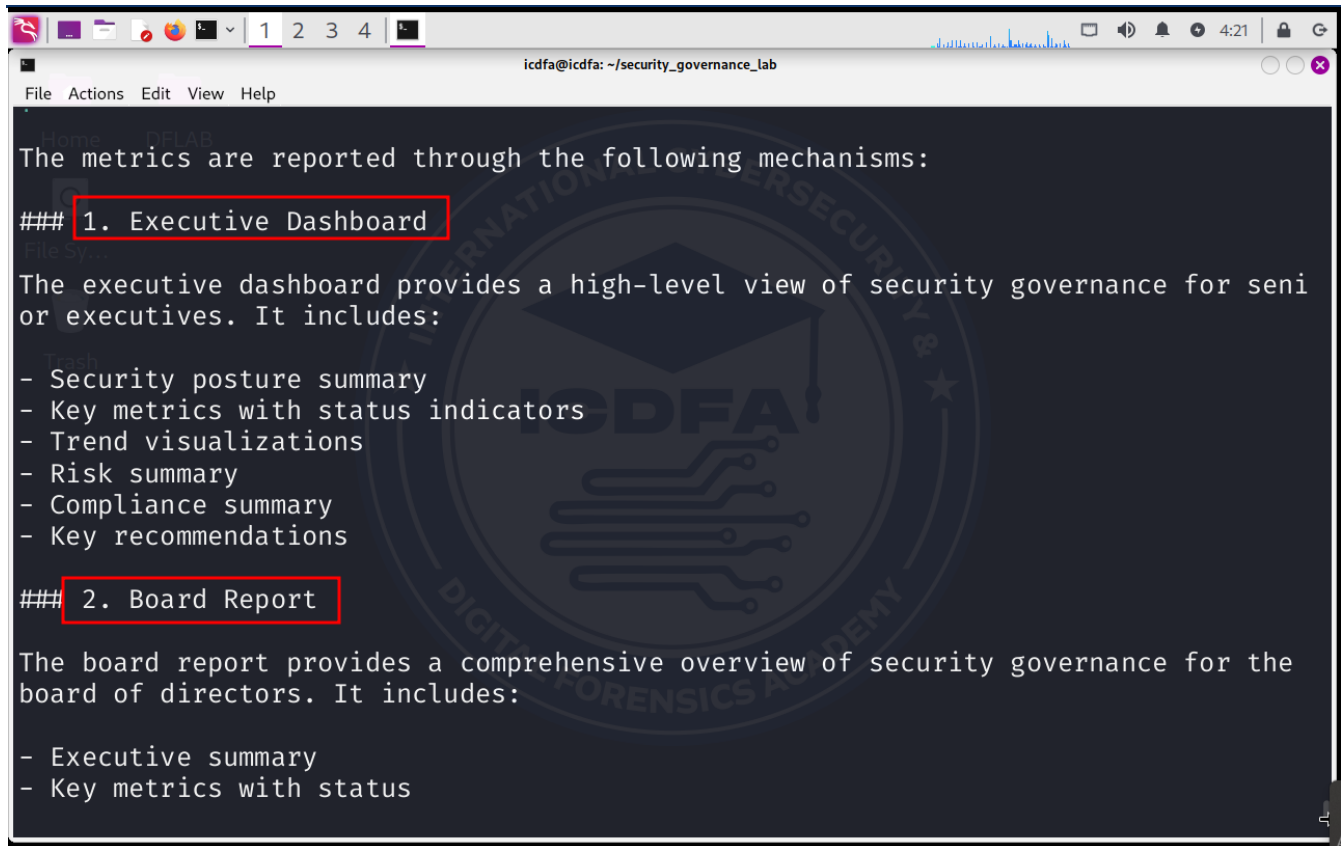
## Metrics Categories

The framework includes metrics in the following categories:

### 1. Implementation Metrics

Implementation metrics measure the extent to which security controls are implemented.

- **Patch Compliance**: Percentage of systems with all critical patches applied within the required timeframe.
- **Security Monitoring Coverage**: Percentage of critical systems covered by security monitoring.
```



```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

The metrics are reported through the following mechanisms:

### 1. Executive Dashboard

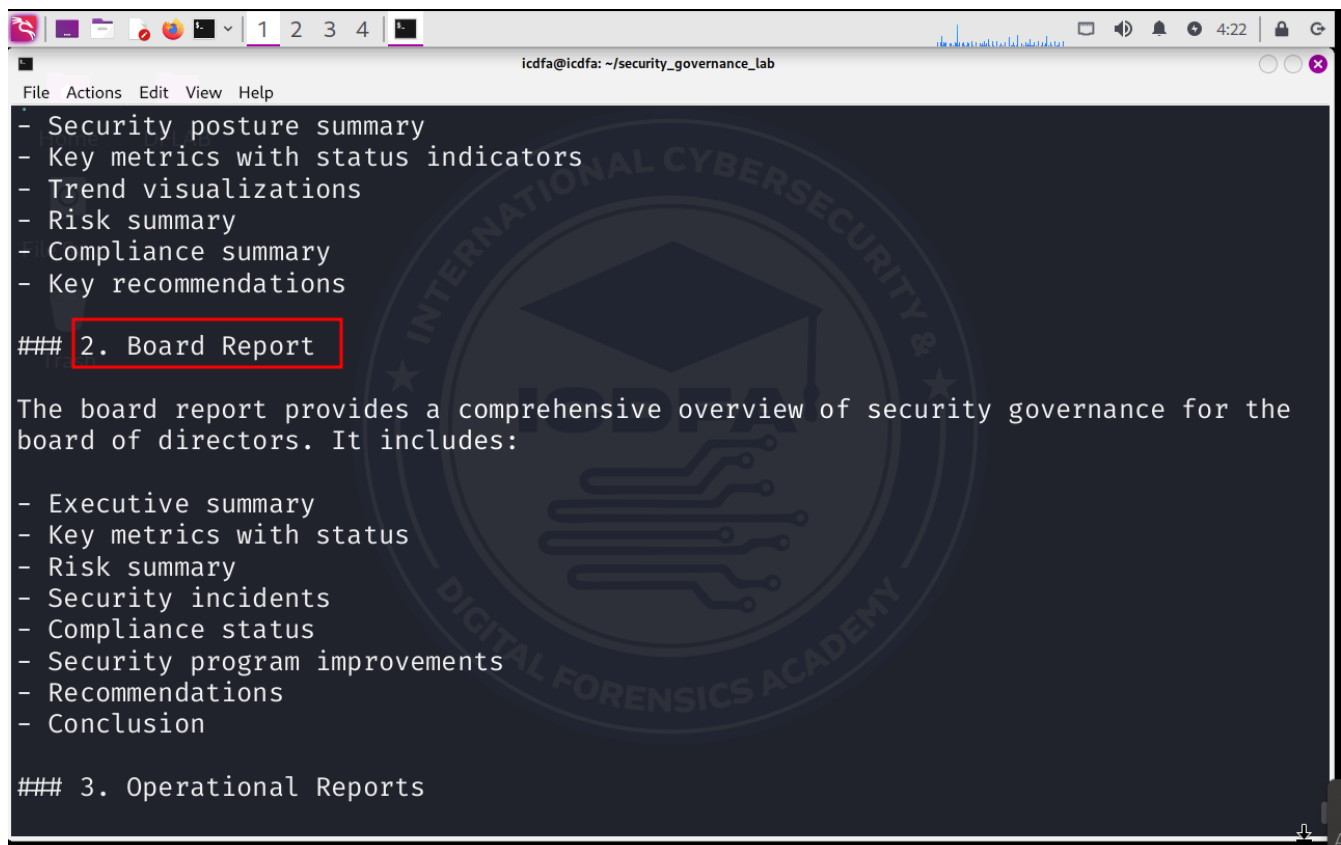
The executive dashboard provides a high-level view of security governance for seni
or executives. It includes:

- Security posture summary
- Key metrics with status indicators
- Trend visualizations
- Risk summary
- Compliance summary
- Key recommendations

### 2. Board Report

The board report provides a comprehensive overview of security governance for the
board of directors. It includes:

- Executive summary
- Key metrics with status
```



```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

- Security posture summary
- Key metrics with status indicators
- Trend visualizations
- Risk summary
- Compliance summary
- Key recommendations

### 2. Board Report

The board report provides a comprehensive overview of security governance for the
board of directors. It includes:

- Executive summary
- Key metrics with status
- Risk summary
- Security incidents
- Compliance status
- Security program improvements
- Recommendations
- Conclusion

### 3. Operational Reports
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

Operational reports provide detailed information for security and IT teams. They include:

- Detailed metrics
- Specific findings
- Action items
- Technical details

## Metrics Lifecycle

The metrics framework follows a lifecycle approach:

1. **Define**: Establish metrics based on security objectives and requirements.
2. **Collect**: Gather data from relevant sources.
3. **Analyze**: Process and analyze the data to calculate metrics.
4. **Report**: Present the metrics to appropriate stakeholders.
5. **Act**: Take action based on the metrics to improve security governance.
6. **Review**: Regularly review and update the metrics framework.

## Continuous Improvement
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

The metrics framework is subject to continuous improvement:

- Regular review of metrics to ensure relevance and effectiveness
- Addition of new metrics as needed
- Retirement of metrics that are no longer valuable
- Adjustment of targets based on changing risk landscape and business requirements

## Conclusion

This security governance metrics framework provides a comprehensive approach to measuring, monitoring, and reporting on security governance. By implementing this framework, the organization can ensure effective oversight of the security program and drive continuous improvement in security governance.
EOF

(icdfa@icdfa)-[~/security_governance_lab]
$
```

4. Create a security governance dashboard user guide:

Create a dashboard user guide

```
cat > dashboard_user_guide.md << 'EOF'
```

Security Governance Dashboard User Guide

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' with a menu bar (File, Actions, Edit, View, Help). The terminal shows the command 'cat > dashboard_user_guide.md << 'EOF'' being executed, followed by the content of the file being created. The content includes a title, an overview paragraph, and a section on accessing the dashboard. A large, faint watermark of the ICDFA logo is visible in the background.

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
$ # Create a dashboard user guide
cat > dashboard_user_guide.md << 'EOF'
# Security Governance Dashboard User Guide

## Overview

This user guide provides instructions for using the Security Governance Dashboard.
The dashboard is designed to provide executives and board members with a clear vi
ew of the organization's security governance posture.

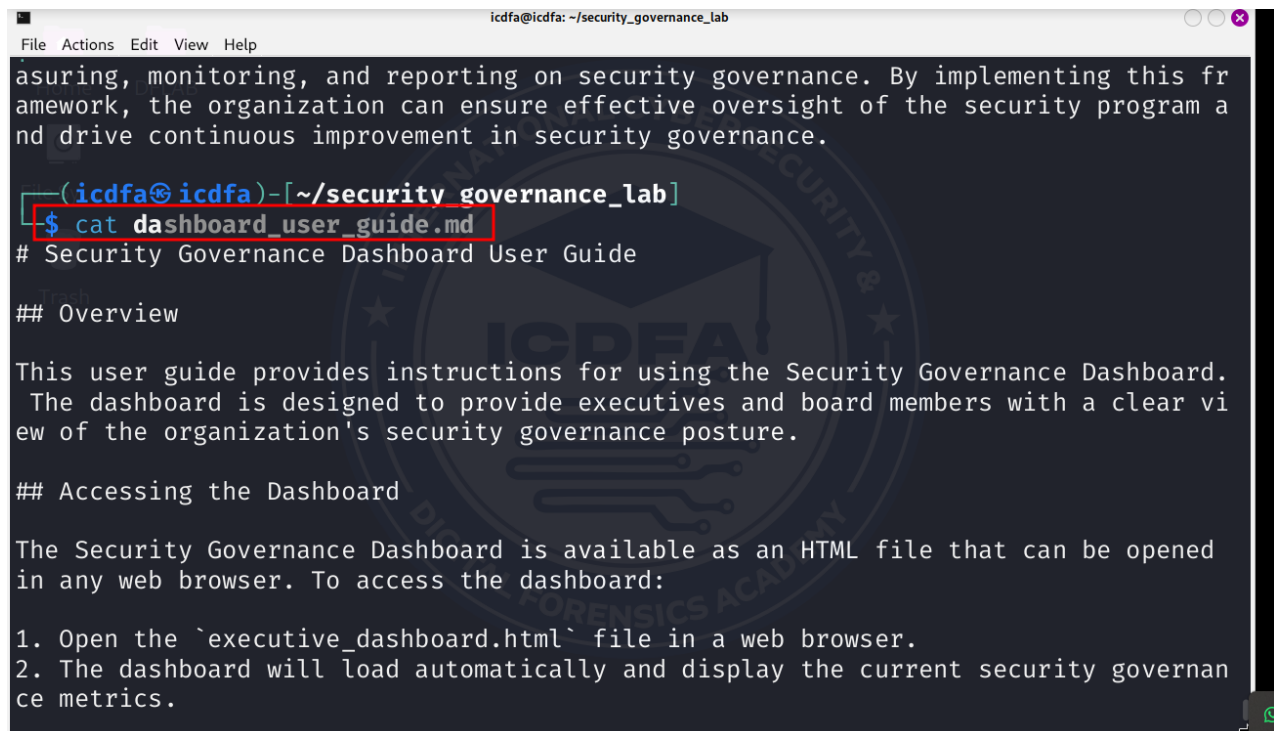
## Accessing the Dashboard

The Security Governance Dashboard is available as an HTML file that can be opened
in any web browser. To access the dashboard:

1. Open the `executive_dashboard.html` file in a web browser.
2. The dashboard will load automatically and display the current security governan
ce metrics.

## Dashboard Sections

The dashboard is organized into the following sections: <....
```

A terminal window titled 'icdfa@icdfa: ~/security_governance_lab' with a menu bar (File, Actions, Edit, View, Help). The terminal shows the command '\$ cat dashboard_user_guide.md' being executed, followed by the content of the file. The content includes an overview paragraph, a section on accessing the dashboard, and a list of steps. A large, faint watermark of the ICDFA logo is visible in the background.

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
$ cat dashboard_user_guide.md
# Security Governance Dashboard User Guide

## Overview

This user guide provides instructions for using the Security Governance Dashboard.
The dashboard is designed to provide executives and board members with a clear vi
ew of the organization's security governance posture.

## Accessing the Dashboard

The Security Governance Dashboard is available as an HTML file that can be opened
in any web browser. To access the dashboard:

1. Open the `executive_dashboard.html` file in a web browser.
2. The dashboard will load automatically and display the current security governan
ce metrics.
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
vulnerability_report_20261009_025837.json
vulnerability_scanner.py
(icdfa@icdfa)-[~/security_governance_lab]
$ cat dashboard_user_guide.md
# Security Governance Dashboard User Guide

## Overview

This user guide provides instructions for using the Security Governance Dashboard.
The dashboard is designed to provide executives and board members with a clear view
of the organization's security governance posture.

## Accessing the Dashboard

The Security Governance Dashboard is available as an HTML file that can be opened
in any web browser. To access the dashboard:

1. Open the `executive_dashboard.html` file in a web browser.
2. The dashboard will load automatically and display the current security governance
metrics.
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
#### 5. Key Recommendations

This section provides recommendations for improving the organization's security governance posture.

## Interpreting the Dashboard

### Metric Status Indicators

Metrics on the dashboard are displayed with status indicators:

- **Green**: Metric is meeting or exceeding the target
- **Yellow**: Metric is below target but within acceptable range
- **Red**: Metric is significantly below target and requires immediate attention

### Trend Visualizations

Trend visualizations show the metric value over time, with a target line indicating the desired value. The trend line helps identify patterns and progress over time.

## Using the Dashboard for Decision Making
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
ting targets
2. **Tracking Progress**: Showing trends over time to track improvement
3. **Prioritizing Actions**: Focusing attention on the most critical issues
4. **Demonstrating Compliance**: Providing evidence of compliance with policies and regulations

## Refreshing the Dashboard

The dashboard is updated monthly with new data. To refresh the dashboard:

1. Run the `governance_metrics.py` script
2. Open the newly generated `executive_dashboard.html` file

## Conclusion

The Security Governance Dashboard provides a powerful tool for monitoring and improving security governance. By regularly reviewing the dashboard, executives and board members can ensure effective oversight of the security program and drive continuous improvement in security governance.

(icdfa@icdfa)-[~/security_governance_lab]
$
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help

(icdfa@icdfa)-[~/security_governance_lab]
$ ls
analyze_governance_failures.py      initialize_governance.py
attack_log.json                    metrics_visualizations
board_report.md                     monitoring
dashboard_user_guide.md            patch_management.py
database_init                       patch_status.json
docker-compose.yml                 __pycache__
equifax_comparison.md              simulate_attack.py
executive_dashboard.html            vulhub-src
governance_controls_report.md       vulnerability_report_20261008_231216.json
governance_data.json               vulnerability_report_20261008_232715.json
governance_failure_report_20261009_013242.json
governance_metrics_framework.md    vulnerability_report_20261009_002023.json
governance_metrics.py              vulnerability_report_20261009_022015.json
governance_oversight.md            vulnerability_report_20261009_023346.json
governance_tracker.py              vulnerability_report_20261009_025038.json
implement_governance_controls.py    vulnerability_report_20261009_025837.json
vulnerability_scanner.py

(icdfa@icdfa)-[~/security_governance_lab]
$
```

```
icdfa@icdfa: ~/security_governance_lab
File Actions Edit View Help
d regulations
## Refreshing the Dashboard

The dashboard is updated monthly with new data. To refresh the dashboard:

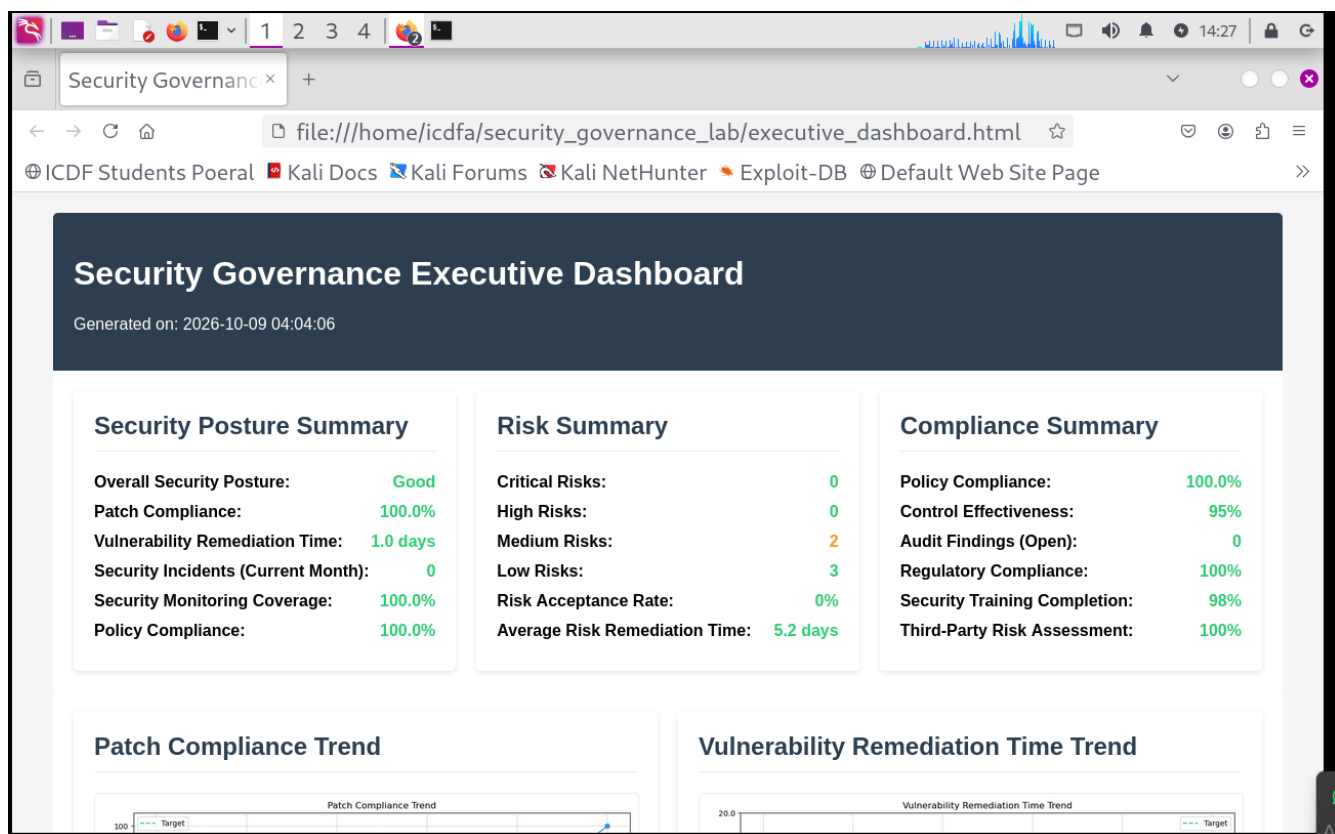
1. Run the `governance_metrics.py` script
2. Open the newly generated `executive_dashboard.html` file

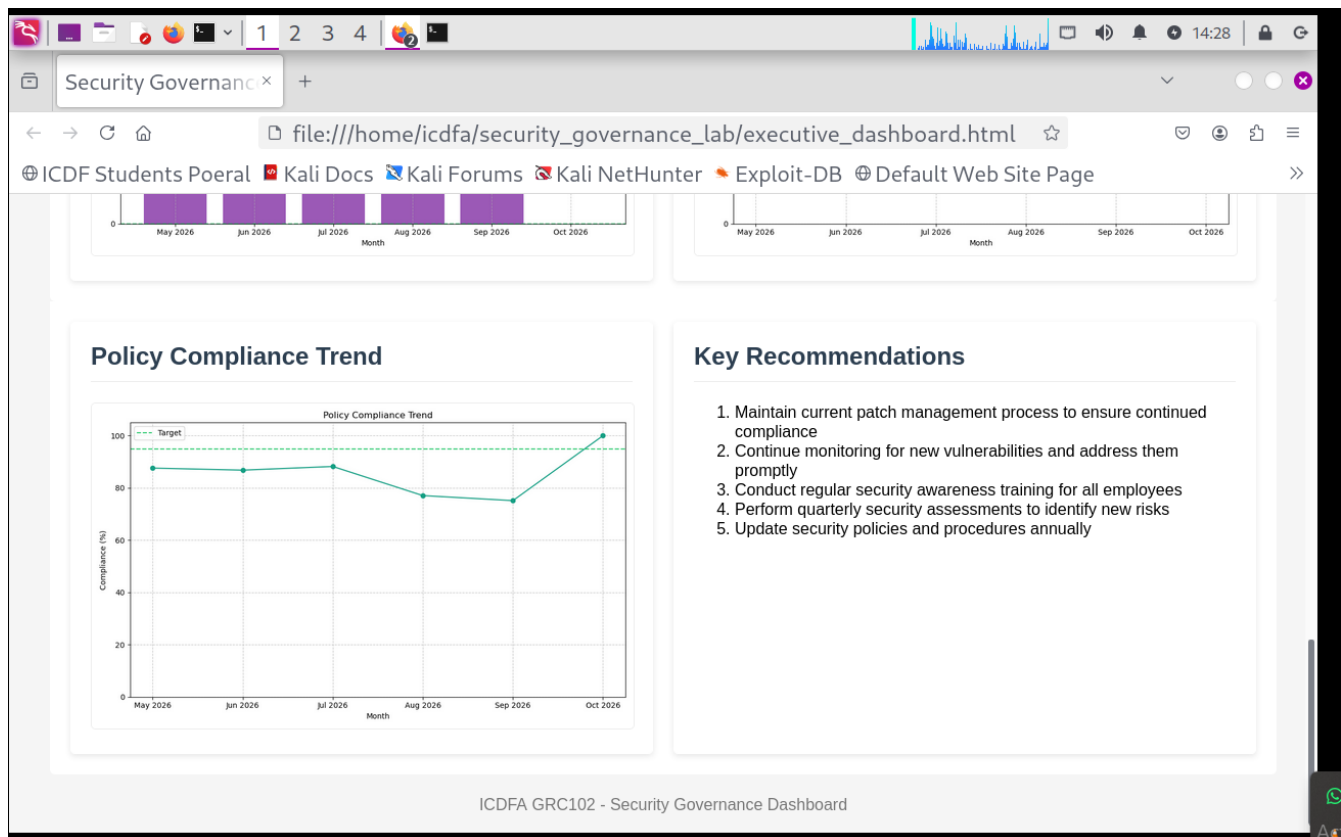
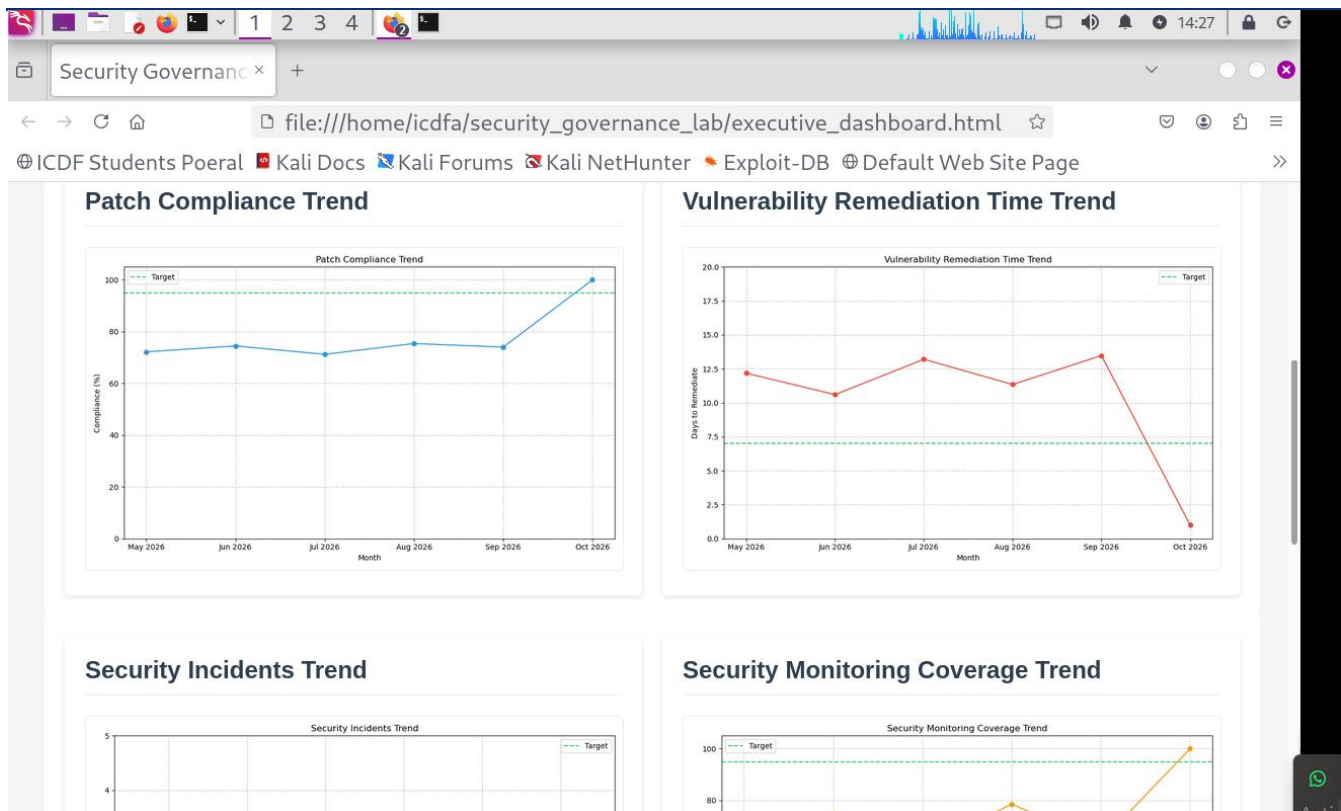
## Conclusion

The Security Governance Dashboard provides a powerful tool for monitoring and improving security governance. By regularly reviewing the dashboard, executives and board members can ensure effective oversight of the security program and drive continuous improvement in security governance.

(icdfa@icdfa)-[~/security_governance_lab]
$ firefox executive_dashboard.html &
[1] 2380677

(icdfa@icdfa)-[~/security_governance_lab]
$
```





How the metrics and reporting mechanisms support effective security governance

Security governance metrics help management understand the organisation's security posture without having to review technical logs and vulnerability scans. Metrics such as Patch Compliance, Vulnerability Remediation Time, and Security Monitoring Coverage turn technical findings into figures that executives and board members can monitor against set targets. This improves visibility and helps leadership identify problems that need attention. In the Equifax breach, the Struts vulnerability was flagged by an internal scan, but the issue was not addressed in time, showing the importance of connecting technical findings to management decisions.

However, this lab also showed that dashboards and reports are only useful when their information is accurate. The reports showed zero critical risks and full regulatory compliance, even though earlier scans still identified critical and high-risk vulnerabilities, and an incident had been recorded. This mismatch shows that reports must be checked against actual system results before they are shared with management. Otherwise, inaccurate figures can create a false sense of security and prevent leaders from recognising risks that need urgent attention.